

Ikigai Masala

The Complete Study Guide

*Understanding constraint-based menu planning,
explained like you are 12.*

What you will learn:

- How OR-Tools turns menu planning into a math puzzle
- How every rule becomes an equation the solver respects
- How data flows from Excel through pools into a finished plan
- How custom per-client rules plug into the system
- How the Flask API, Streamlit UI, and Supabase fit together
- How to safely change, debug, and extend the codebase

Table of Contents

PART A — Core Concepts

1. What problem does this app solve?
2. What is OR-Tools?
3. Turning menus into a math puzzle
4. How rules become equations
5. The two-phase rule system
6. Hard vs soft rules
7. The big picture flow

PART B — File Architecture

8. Top-level project layout
9. The four-floor mental model
10. Folder-by-folder tour

PART C — The Data Pipeline

11. Excel reading and column mapping
12. Data cleansing
13. Pool building and slot expansion

PART D — The Solver Deep Dive

14. Cell architecture
15. Candidate sampling with priority
16. Building the CP-SAT model step by step
17. The objective function
18. Multi-restart strategy
19. Solution extraction

PART E — All Rules as Equations

20. The rule translation master class
21. End-to-end: how one menu gets generated

PART F — Individual Rule Deep Dives

- 22. Cooldown rules
- 23. Theme rules
- 24. Color rules
- 25. Coupling rule
- 26. Curd side rule
- 27. Premium and unique items
- 28. Cuisine and non-veg rules

PART G — Custom Per-Client Rules

- 29. How custom rules are stored and loaded
- 30. Ingredient ban rule
- 31. Item frequency rule
- 32. Slot day restriction rule
- 33. Adding your own custom rule

PART H — The Web Layer

- 34. Flask API overview
- 35. The /plan endpoint, end-to-end
- 36. /regenerate and /save
- 37. Validation and error handling

PART I — Concurrency and Performance

- 38. The solver gate
- 39. Rate limiting
- 40. Caching at every layer

PART J — Authentication and Security

- 41. Roles and permissions
- 42. Password hashing and bearer tokens
- 43. Cookie-based session restoration
- 44. Optimistic concurrency

PART K — Database Layer

- 45. Supabase schema explained

-
- 46. The live-read pattern
 - 47. Schema migration safety

PART L — Streamlit Frontend

- 48. The main app flow
- 49. Customisation editor
- 50. User manager UI

PART M — Operations and Observability

- 51. Structured logging
- 52. In-process metrics
- 53. Health checks

PART N — Testing and CI

- 54. Test architecture
- 55. Fake Supabase
- 56. CI pipeline

PART O — Final Recap and Mental Models

- 57. The whole system, one page
- 58. Common questions and pitfalls

Part A — Core Concepts

Welcome! This first part is the foundation. Master these six chapters and the rest of the book will fall into place. We will go slowly and use lots of pictures, analogies, and tiny code snippets. No prior experience with optimization solvers is assumed.

Chapter 1 — What Problem Does This App Solve?

Imagine you run a cafeteria that serves lunch to office workers at companies like Rippling, Stripe, Tekion, and many more. Every week you need to plan a menu for each company. A menu looks like this:

Slot	Mon	Tue	Wed	Thu	Fri
Rice	Jeera Rice	Fried Rice	Veg Biryani	Sambar Rice	Pulao
Bread	Naan	Chapati	Roti	Dosa	Paratha
Veg Dry	Aloo Gobi	Manchurian	Bhindi Masala	Beans Poriyal	Bhindi Masala
Dal	Dal Tadka	Tomato Dal	Sambar	Drumstick Sambar	Yellow Dal

Now here is the tricky part: you cannot just pick random items! Every menu has to follow many rules at the same time.

The rules a real menu has to obey

- Do not repeat the same item within 20 days — diners get bored.
- Each day must have at least four different colors of food — visual variety matters.
- Tuesday is Chinese day, Wednesday is Biryani day, Thursday is South Indian, Friday is North Indian, and Monday mixes south and north.
- If a client says “no mushroom”, never serve mushroom to that client.
- Some clients only allow non-veg dishes on specific weekdays.
- Premium items like paneer dishes are limited — at most two per week.
- If you serve a rice-based bread, you must pair it with a soupy rice and a deep-fried starter.
- If today's rice is biryani, today's curd side must be raita, not plain curd.
- There are roughly 20 more rules just like these.

■ Mental picture

Think of it like a giant Sudoku puzzle. Every cell of the grid (one slot per day) needs a value, and every value has to satisfy multiple intersecting rules. Doing it by hand for eighteen different clients every week would take days. So we built a computer to do it.

■ Key Takeaway

Ikigai Masala is a constraint-based menu planner. You give it a client, a start date, and a number of days; it produces a fully-rule-compliant menu in seconds.

Chapter 2 — What is OR-Tools?

OR-Tools is a free toolkit made by Google that solves constraint puzzles. It is like having a very patient mathematician living inside your computer who is willing to try millions of combinations until everything fits.

The Sudoku connection

If you have ever played Sudoku, you already understand the core idea. In Sudoku you fill a 9x9 grid with the digits 1 through 9 such that every row, every column, and every 3x3 box contains all nine digits exactly once. A computer Sudoku solver tries different placements until it finds one where every rule is satisfied.

OR-Tools does exactly that, but for any problem you can describe as (variables) + (rules they must follow) + (something to maximize). It is not limited to Sudoku.

CP-SAT — the specific tool we use

Inside OR-Tools there are several solvers. We use CP-SAT, which stands for Constraint Programming — Satisfiability. Don't worry about the name. Just remember: CP-SAT is the puzzle-solver inside OR-Tools.

```
from ortools.sat.python import cp_model

model = cp_model.CpModel()
# ... we tell it variables and rules ...
solver = cp_model.CpSolver()
status = solver.Solve(model)
```

Three things happen here: we build a model object, we add things to it, and then we ask a solver to find values that satisfy everything. The rest of this book is essentially a tour of what we add to the model and why.

Chapter 3 — Turning Menus Into a Math Puzzle

This is the single most important idea in the whole codebase. If you remember nothing else from this book, remember this chapter.

The big idea: one switch per possibility

Imagine Monday's Rice slot. We have four candidate dishes:

- Jeera Rice
- Lemon Rice
- Tomato Rice
- Pulao

Instead of saying “pick one”, the solver creates four light switches — one for each candidate:

```
Monday-Rice-JeeraRice = OFF (0)
Monday-Rice-LemonRice = OFF (0)
Monday-Rice-TomatoRice = OFF (0)
Monday-Rice-Pulao = OFF (0)
```

Each switch is a boolean variable: it can be 0 (off) or 1 (on). Then we add a constraint that says “exactly one switch in this group must be on”:

```
JeeraRice + LemonRice + TomatoRice + Pulao = 1
```

If the solver flips LemonRice to 1 and leaves the others at 0, the equation balances ($0+1+0+0 = 1$). That means Monday gets Lemon Rice.

■ Picture in your head

Picture a classroom row of light switches on a wall. The teacher says “exactly one of you turns yours on”. Each kid is a candidate dish; whoever turns their switch on is the chosen item. The solver is the teacher figuring out which switches across hundreds of rows lead to a configuration where every classroom rule is happy.

The real code

Here is the core of variable creation in `src/solver/menu_solver.py`:

```
for j in range(len(cell.cand_df)):
    row = cell.cand_df.iloc[j]
    var = model.NewBoolVar(f'x_d{di}_{slot_id}_{j}')
    x_vars.append(var)
```



```
cand_rows.append(row)
# ... track this var for color, premium, theme constraints ...

# Exactly one candidate per cell
model.Add(sum(x_vars) == 1)
```

NewBoolVar creates a fresh on/off switch with a unique name like `x_d0_rice_3` (day 0, rice slot, third candidate). **model.Add(sum(x_vars) == 1)** is the “exactly one wins” rule.

■ **Tip**

For a five-day plan with eleven slots and roughly fifty candidates per slot, the solver creates around $5 \times 11 \times 50 = 2,750$ boolean switches. Then it has to find an on/off pattern across all of them that satisfies every rule. That is why it is so much faster than a human.

Chapter 4 — How Rules Become Equations

Now we have switches. How do real-world rules become equations the solver understands? Let's walk through three examples — short for now; Part E will cover every rule in detail.

Example 1 — At most one premium item per day

Premium items are flagged in the data with `is_premium_veg = 1`. Suppose Monday has three premium candidate switches:

- Mon-VegGravy-PaneerButter (premium)
- Mon-VegDry-PaneerTikka (premium)
- Mon-Starter-PaneerKebab (premium)

The rule becomes one equation:

```
PaneerButter + PaneerTikka + PaneerKebab <= 1
```

Even though all three are valid candidates, the solver can pick at most one. The other two switches will be forced off. That is real production code in `src/menu_rules/premium_menu_rule.py`:

```
model.Add(sum(lits) <= self.max_per_day)
```

Example 2 — No repeats across the week

If *Jeera Rice* is a candidate on Monday, Tuesday, and Wednesday, we end up with three switches with the same item base name. The rule “use this item at most once” becomes:

```
Mon_JeeraRice + Tue_JeeraRice + Wed_JeeraRice <= 1
```

Real code in `unique_items_menu_rule.py`:

```
for item_base, vars_ in item_to_vars.items():
    if item_base not in repeatable:
        model.Add(sum(vars_) <= 1)
```

Example 3 — Tuesday must be Chinese

This rule is special: we don't write a math equation for it. Instead we filter the candidate pool BEFORE creating switches. For Tuesday's Rice slot we keep only items where `is_chinese_fried_rice = 1`:

```
Original pool: [Jeera, Lemon, FriedRice, Schezwan, Pulao]
                | filter
                v
Tuesday Rice: [FriedRice, Schezwan] # only Chinese
```

Now no matter what the solver picks for Tuesday's rice, it will be Chinese — because those are the only switches that exist. This is called pre-filtering, and it is faster than letting the solver discover the rule via constraints.

Chapter 5 — The Two-Phase Rule System

Every rule in this codebase participates in two phases, like cleaning your room before friends come over. First you throw out the obvious trash, then you organize what is left.

Phase 1 — Pre-filter (before making switches)

Some rules are easy: just remove bad candidates from the pool. No math, no solver — just list operations on a pandas DataFrame.

- Banned ingredient (mushroom)? Remove all mushroom items.
- Used in the last 20 days? Remove from today's candidates.
- Tuesday is Chinese day? Remove non-Chinese rice items.
- Rice-bread used three days ago, gap is ten days? Remove rice-bread items today.

Phase 2 — CP-SAT (after making switches)

Other rules need real math because they involve relationships among variables we have already created.

- "At most 1 premium per day" — needs to count switches.
- "Rice-bread implies liquid rice" — if-then logic.
- "At least 4 distinct colors today" — needs counting distinct values.

The contract every rule follows

Every rule inherits from **BaseMenuRule** and may override one or more of three methods:

```
class BaseMenuRule:

    def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
        # Phase 1: filter the candidate list
        return pool # default = no filtering

    def apply(self, model, variables, menu_data, context):
        # Phase 2: add hard math equations to the solver
        pass

    def get_objective_terms(self, model, context):
        # Phase 3: add bonus/penalty preferences (not strict rules)
        return []
```

pre_filter_pool handles Phase 1. **apply** adds hard constraints. **get_objective_terms** adds soft preferences (we'll cover those next chapter).

■ **Tip**

Why two phases? Filtering is cheap (just list operations). Adding math equations is expensive (the solver has to think about each one). So we do as much as we can in Phase 1 to keep the solver's job manageable.

Chapter 6 — Hard vs Soft Rules: Strict Mom vs Cool Aunt

Not all rules are equally important. The codebase formally classifies rules into HARD and SOFT severity.

Hard rules — strict mom

“This MUST happen, or no menu at all.” If a hard rule cannot be satisfied, the solver returns INFEASIBLE and we have no plan. Examples:

- No banned ingredients
- No item repeats within the horizon
- Max one premium item per day
- At least four distinct colors per day

Soft rules — cool aunt

“It would be nice if this happens, but we will not crash if it doesn't.” These rules contribute terms to the **objective function** — a single number the solver tries to maximize.

- Prefer theme-matching starters: bonus of +1,000,000 per match
- Avoid re-using a recent week verbatim: small penalty
- Penalty when a non-theme fallback is used: -2,000,000

In code (**base_menu_rule.py**):

```
class MenuRuleSeverity(Enum):
    HARD = "hard" # break = solve fails
    SOFT = "soft" # break = just lower score
```

How the objective works

Imagine you're playing a video game where each choice gives or costs points. The solver picks the combination of switches with the highest total score:

```
Theme starter chosen on Monday : +1,000,000
Theme starter chosen on Tuesday: +1,000,000
Non-theme fallback on Wed : -2,000,000
Avoiding repeated week sig : +small
```

```
-----
Total objective = 0 (in this case)
```

We want this number as big as possible. The solver searches for the configuration of switches that maximizes it.

Real code:

```
model.Maximize(sum(obj_terms))
```

■ Key Takeaway

Hard rules are constraints. Soft rules are preferences encoded as objective terms. The solver guarantees hard rules and tries its best on soft ones. If a soft rule's evaluation throws an exception, the solve still finishes and the failure is reported in the response.

Chapter 7 — The Big Picture Flow

Putting it all together: when a user clicks "Generate Menu Plan" in the Streamlit UI, this is what happens behind the scenes.

```
Step 1. USER clicks Generate in Streamlit (app.py)
      |
      | HTTP POST /api/v1/plan with bearer token
      v
Step 2. FLASK API receives request (api/app.py)
      - Check authentication via require_api_auth
      - Check rate limit (token bucket, e.g. 10 plans/min)
      - Wait in solver gate queue (max 2 concurrent solves)

Step 3. LOAD client config from Supabase
      "Tekion uses menu_cat_1, version 3, theme overrides ..."

Step 4. LOAD all menu items from Excel
      ~530 items with cuisine, color, premium flags

Step 5. BUILD pools – sort items by base slot
      rice_pool: [JeeraRice, LemonRice, ...]
      bread_pool: [Naan, Chapati, ...]

Step 6. LOAD rules
      14 generic rules + 3 Tekion-specific custom rules

Step 7. LOAD history from Supabase (last ~45 days)
      "We served these items recently" -> banned set per date

Step 8. SOLVE with CP-SAT (the magic)
      a) Pre-filter every (day, slot) pool
      b) Build _Cell objects, sample candidates
      c) Create boolean switches (~2,750 of them)
      d) Add hard constraint equations
      e) Add soft objective terms
      f) Call solver.Solve() with multi-restart strategy
      g) Read selected switches back into a plan dict

Step 9. FORMAT result as JSON
      { "2026-05-05": { "items": { "rice": "jeera_rice(Y)", ... } } }

Step 10. STREAMLIT renders the colored table for the user
```

■ Where we go next

Every menu generation walks this same path. Once you hold this picture in mind, the rest of the codebase is just "what does each step do in more detail?". Parts B through F answer exactly that.

Part B — File Architecture

Before diving into the engine, let's take a guided tour of the building. Knowing what lives in which folder makes everything else easier — when you read about a class, you'll know where to find it.

Chapter 8 — Top-Level Project Layout

From the repository root, the project tree looks like this. Most everything you'll touch lives inside the **ikigai_masala-main/** folder.

```
ikigai_masala-main/
|
|-- app.py <- FRONT DOOR (Streamlit entry point)
| auto-starts Flask, handles login
|
|-- api/ <- BACKEND (Flask REST server)
| |-- app.py all 14 endpoints
| |-- auth.py bearer token signing
| |-- concurrency.py solver queue (max 2 concurrent)
| |-- config.py constants, env vars, paths
| |-- logging_config.py structured logs + request IDs
| |-- metrics.py in-process counters
| |-- rate_limit.py token bucket rate limiter
|
|-- src/ <- THE BRAIN (business logic)
| |-- constants.py slot names, color codes
| |-- db.py Supabase singleton client
| |
| |-- solver/ THE PUZZLE SOLVER
| | |-- menu_solver.py main CP-SAT solver
| | |-- solver_context.py bundle passed to rules
| | |-- solution_formatter.py output -> JSON
| | |-- regenerator.py "replace these cells"
| | |-- _helpers.py shared utilities
| |
| |-- menu_rules/ ALL 19 RULES
| | |-- base_menu_rule.py parent class
| | |-- menu_rule_loader.py JSON -> rule objects
| | |-- ... 17 more rule files
| |
| |-- preprocessor/ DATA PIPELINE
| | |-- excel_reader.py
| | |-- column_mapper.py
| | |-- data_cleanser.py
| | |-- pool_builder.py
| | |-- theme_filter.py
| |
| |-- client/
| | |-- client_config.py per-client settings (Supabase)
```

```

| |
| |-- history/
| |-- history_manager.py past menus + cooldowns
|
|-- ui/ <- FRONTEND HELPERS
| |-- api_client.py HTTP client
| |-- backend_probe.py port picker + health checks
| |-- formatters.py pretty-print items, themes
| |-- styles.py all CSS
|
|-- customisation/ <- ADMIN EDITOR
| |-- main.py editor dispatcher
| |-- slot_editor.py toggle categories
| |-- multi_slot_editor.py set frequency
| |-- theme_editor.py per-day theme override
|
|-- user_authentication/ <- LOGIN SYSTEM
| |-- models.py User dataclass + roles
| |-- auth_manager.py bcrypt + Supabase
| |-- session.py Streamlit session helpers
| |-- cookie_store.py persistent token in cookie
| |-- login_ui.py login form
| |-- user_manager_ui.py admin: create/delete users
|
|-- data/
| |-- raw/menu_items.xlsx THE ONTOLOGY (~530 dishes)
| |-- configs/
| |-- indian_menu_rules.json generic rules config
| |-- client_rules.json per-client custom rules
| |-- clients.json legacy seed data
|
|-- scripts/ <- ONE-OFF UTILITIES
| |-- create_tables.sql
| |-- create_history_tables.sql
| |-- create_users_table.sql
| |-- seed_supabase.py
| |-- seed_admin.py
|
|-- tests/ <- 477 TESTS
| |-- conftest.py shared fixtures
| |-- fake_supabase.py in-memory DB for tests
| |-- test_*.py all test files
|
|-- docs/ <- DOCUMENTATION
| |-- setup.md
| |-- architecture.md
| |-- api.md
| |-- operations.md

```

Chapter 9 — The Four-Floor Mental Model

All those folders fit into a clean four-story building. Each floor has one job, and each floor only talks to the floor below it. This is the mental model that makes the codebase feel small.

```
+-----+
| FLOOR 4 – UI Layer |
| app.py, ui/, customisation/, user_auth/*_ui.py |
| &quot;What the user sees and clicks&quot; |
+-----+
| HTTP
| v
+-----+
| FLOOR 3 – API Layer |
| api/ |
| &quot;Translates clicks into business operations&quot; |
+-----+
| function calls
| v
+-----+
| FLOOR 2 – Business Logic |
| src/solver/, src/menu_rules/, src/preprocessor/ |
| &quot;The actual menu-planning brain&quot; |
+-----+
| read/write
| v
+-----+
| FLOOR 1 – Data Layer |
| src/client/, src/history/, src/db.py, Supabase, Excel |
| &quot;Where data lives&quot; |
+-----+
```

■ Analogy

Like a hotel: the lobby (UI) takes your order, the front desk (API) tells the kitchen (Logic) what to cook, and the kitchen pulls ingredients from the pantry (Data). The lobby never goes into the pantry directly. That separation is what keeps the codebase clean.

Chapter 10 — Folder-by-Folder Tour

Here is a one-line summary of every important file. Bookmark this page — you'll come back to it often.

api/ — The REST backend

File	What it does
app.py	All 14 endpoints (/plan, /save, /clients, ...). The largest file in this folder.
auth.py	Issues and verifies signed bearer tokens via itsdangerous.
concurrency.py	The solver_gate decorator: queues solves, caps at 2 concurrent.
config.py	Path constants, env-var validation, timezone resolution.
logging_config.py	JSON formatter, request_id ContextVar, log dict-config.
metrics.py	Tiny in-process counter library, Prometheus-style key format.
rate_limit.py	Token-bucket rate limiter with per-principal buckets.

src/solver/ — The CP-SAT brain

File	What it does
menu_solver.py	MenuSolver + SolverConfig + the _Cell class. The heart of everything.
solver_context.py	TypedDict describing what rules receive in apply().
solution_formatter.py	Turns the solver's plan dict into the JSON the UI consumes.
regenerator.py	Locks unchanged cells, asks the solver to redo only selected ones.
_helpers.py	Shared utilities: weekday_type, theme_label, strip_color_suffix.

src/menu_rules/ — Where every rule lives

File	Rules contained
base_menu_rule.py	BaseMenuRule abstract class + MenuRuleSeverity enum.
menu_rule_loader.py	Reads JSON config, instantiates rule objects.
cuisine_menu_rule.py	CuisineMenuRule
color_rules.py	ColorVariety, ColorPairing, WelcomeDrinkColor
cooldown_rules.py	ItemCooldown, RiceBreadGap, WeekSignatureCooldown
coupling_menu_rule.py	CouplingMenuRule (the deep-fried chain)
curd_side_menu_rule.py	CurdSideMenuRule
nonveg_rules.py	NonvegBiryaniWeekly, NonvegDryPreference
premium_menu_rule.py	PremiumMenuRule
theme_rules.py	ThemeDay, ThemeSlotFilter, ThemeStarterPreference, ThemeFallbackPenalty
unique_items_menu_rule.py	UniqueltemsMenuRule
ingredient_ban_rule.py	[Custom] IngredientBanRule
item_frequency_rule.py	[Custom] ItemFrequencyRule
slot_day_restriction_rule.py	[Custom] SlotDayRestrictionRule

src/preprocessor/ — Data pipeline

File	What it does
excel_reader.py	Loads the .xlsx ontology, hands off to ColumnMapper.
column_mapper.py	Detects column aliases, normalizes flags, computes key_eff.
data_cleanser.py	Removes duplicates, fills missing fields, casts flag types.
pool_builder.py	Splits the cleaned DataFrame into per-slot pools. Handles slot expansion.
theme_filter.py	Static theme filtering used by the legacy path; ThemeSlotFilterRule replaced most of it.

ui/ — Streamlit-side helpers

File	What it does
api_client.py	MenuApiClient — wraps HTTP calls with retry-on-transient-5xx.
backend_probe.py	Picks an open port for the auto-spawned Flask backend.
formatters.py	format_item_html, theme labels, slot ordering — everything the table renders.
styles.py	All CSS as one big constant. ~700 lines of dark-theme styling.

user_authentication/ — Login system

File	What it does
models.py	User dataclass; ROLE_SUPER_ADMIN, ROLE_ADMIN, ROLE_USER constants.
auth_manager.py	Supabase-backed CRUD; bcrypt hashing; legacy SHA-256 migration.
session.py	Streamlit session_state helpers: login_user, logout_user, current_user.
cookie_store.py	Persists bearer token in a browser cookie for 12 hours.
login_ui.py	The login form widget.
user_manager_ui.py	Admin UI for creating/deleting users.

data/ — Inputs and configs

File	What it contains
raw/menu_items.xlsx	The ontology. ~530 rows, one per dish, with cuisine, color, flags.
configs/indian_menu_rules.json	Generic rule list applied to every client.
configs/client_rules.json	Per-client custom rules (the topic of Part G).
configs/clients.json	Legacy seed data — kept only for empty-DB bootstrap.

■ Key Takeaway

Memorize the four-floor model. When you read about MenuSolver, you know it's on Floor 2. When you see CookieController, it's Floor 4. When a request crosses floors, the file paths in this tour tell you exactly which file to open. Knowing where things live is half the battle when learning a codebase.

Part C — The Data Pipeline

Before the solver can do anything, it needs *candidates* — clean, categorized, normalized lists of dishes. This part follows the data from a raw Excel file all the way to the per-slot pools the solver consumes.

Chapter 11 — Excel Reading and Column Mapping

The ontology lives at **data/raw/menu_items.xlsx**. About 530 rows. Each row is one dish, with about 25 columns describing cuisine, color, key ingredient, and a bunch of boolean flags.

Why the column mapper exists

Different versions of the source spreadsheet use different column names. Sometimes `'item'`, sometimes `'menu_items'`, sometimes `'menu_item'`. Sometimes `'color'`, sometimes `'item_color'`, sometimes `'colour'` (British spelling). Hard-coding one set of names would break every time the source file is regenerated.

ColumnMapper in **src/preprocessor/column_mapper.py** solves this by detecting aliases. It has a dictionary of canonical names with their accepted variants:

```
REQUIRED_COL_ALIASES = {
    'item': ['item', 'menu_items', 'menu_item'],
    'course_type': ['course_type', 'course', 'slot'],
    'cuisine_family': ['cuisine_family', 'cuisine_family_', 'cuisine'],
    'item_color': ['item_color', 'colour', 'color', 'color_group'],
    'key_ingredient': ['key_ingredient', 'keyingredient', 'ingredient_key'],
    'sub_category': ['sub_category', 'subcategory'],
}
```

The flow: detect, validate, apply

When **ExcelReader.read()** is called:

- Pandas loads the .xlsx into a DataFrame.
- **ColumnMapper.detect()** walks every required canonical name and finds whichever column variant is present.
- **ColumnMapper.validate()** raises **ValueError** if 'item' or 'course_type' is missing.
- **ColumnMapper.apply()** does five things in one pass: rename to canonical, fill missing optional columns, normalize text/colors, normalize flag values, and compute `key_eff`.

Normalizing colors

Colors come in messy: *Red*, *RED*, *Light Green*, *NA*, *nan*, blank cells. The mapper unifies them:

```
def _norm_color(x) -> str:
    s = _norm_str(x).replace(' ', '_')
    if s in ('', 'na', 'nan', 'null', 'none', 'unknown', 'unk'):
        return 'unknown'
    return s

# 'Red' -> 'red'
# 'Light Green' -> 'light_green'
# 'NA' -> 'unknown'
# nan -> 'unknown'
```

Normalizing flags

Boolean flag columns (**is_premium_veg**, **is_chinese_starter**, etc.) come in as 1/0, "y"/"n", "yes"/"no", or blank. The mapper coerces every cell to 0 or 1:

```
def _to_bool01(x) -> int:
    if pd.isna(x):
        return 0
    if isinstance(x, (int, float)):
        return int(x != 0)
    return 1 if str(x).strip().lower() in ('1', 'y', 'yes', 'true', 't') else 0
```

The derived key_eff column

Some uniqueness rules cannot rely on the raw item name. For example, "chicken biryani (Hyderabadi)" and "chicken biryani (Lucknowi)" are different items but they share the key ingredient "chicken". The **key_eff** column joins **key_ingredient** + **cuisine_family** + **sub_category** for those special cases (chicken, egg, rice, biryani) so cooldown/uniqueness can dedupe at the right granularity.

■ Tip

key_eff is computed once during preprocessing. Anywhere downstream that needs "is this dish equivalent to that dish" can compare **key_eff** directly. No re-derivation in rules.

Chapter 12 — Data Cleansing

After ColumnMapper, the DataFrame still has problems: duplicate rows, rows with empty item names, NaN values in flag columns. **DataCleanser** in `src/preprocessor/data_cleanser.py` tidies everything up.

What it does

- Drops duplicates based on `item_id` (or `item` if `item_id` is missing).
- Fills missing text fields with empty strings so rules don't crash on NaN.
- Removes rows where 'item' or 'course_type' is empty (no point keeping them).
- Forces every `is_*` flag column to int type, NaN -> 0.

Why this is a separate class

Cleaning logic could live inside ColumnMapper, but separating it keeps responsibilities clear: ColumnMapper is about *schema*, DataCleanser is about *quality*. Tests can exercise either in isolation.

```
df_raw = ExcelReader(path).read() # Excel -> raw DF
df_normalized = ColumnMapper().detect(df_raw).apply(df_raw) # canonical schema
df_clean = DataCleanser(df_normalized).clean() # ready to use
```

■ Analogy

ColumnMapper is the friendly translator who renames columns into a language the rest of the system understands. DataCleanser is the janitor who throws out duplicates and dusts off the NaN cells before the solver opens the door.

Chapter 13 — Pool Building and Slot Expansion

Now we have a clean, normalized DataFrame of ~530 dishes. The solver thinks in terms of *slots* (rice slot, bread slot, dessert slot, etc.), not one giant table. **PoolBuilder** splits the DataFrame into per-slot pools.

Simple 1-to-1 mappings

Most slots have a direct relationship to **course_type**:

```
_SIMPLE_MAPPING = {
    'welcome_drink': {'welcome_drink', 'infused_water'},
    'soup': {'soup'},
    'salad': {'salad'},
    'starter': {'starter'},
    'bread': {'bread'},
    'rice': {'rice'},
    'healthy_rice': {'healthy_rice', 'healthy rice', 'healthy-rice'},
    'dal': {'dal'},
    'veg_gravy': {'veg_gravy'},
    'veg_dry': {'veg_dry'},
    'nonveg_main': {'nonveg_main'},
    'curd_side': {'curd_side'},
    'dessert': {'dessert'},
}

pools[slot] = df[df['course_type'].isin(course_types)]
```

The sambar/rasam split

Some entries in the source data have `course_type = "sambar/rasam"` as one combined category. They have to be split into the correct individual slot based on whether the item name contains `"rasam"`:

```
is_rasam_text = df['item'].str.contains('rasam', na=False)

pools['rasam'] = df[
    (df['course_type'] == 'rasam') |
    ((df['course_type'] == 'sambar/rasam') & is_rasam_text)
]
pools['sambar'] = df[
    (df['course_type'] == 'sambar') |
    ((df['course_type'] == 'sambar/rasam') & ~is_rasam_text)
]
```

Slot expansion — why veg_dry can become two slots

Some clients want two veg dry items per day. We can't put two items into the same slot — “exactly one” is the contract. Instead, we expand the base slot name into numbered variants:

```
slot_counts = {'veg_dry': 2, 'rice': 1, 'bread': 1}

_expand_slots_in_order(['rice', 'veg_dry', 'bread'], slot_counts)
# returns ['rice', 'veg_dry__1', 'veg_dry__2', 'bread']
```

Both **veg_dry__1** and **veg_dry__2** share the same base pool, but each has its own set of switches and its own “exactly one” constraint. The unique items rule (Chapter 27) prevents both expanded slots from picking the same dish on the same day.

■ Tip

When you write a new rule, decide whether it operates on the base slot (**veg_dry**) or the expanded slot id (**veg_dry__2**). Most rules use base — that is what gets passed to **pre_filter_pool()**. **NonvegDryPreferenceRule** is the rare exception that cares about the slot number for a fall-back behavior.

Validation

After all the slicing, PoolBuilder checks that every base slot has at least one item:

```
for slot in BASE_SLOT_NAMES:
    if slot not in pools or len(pools[slot]) == 0:
        raise ValueError(f"Slot '{slot}' has 0 items after mapping.")
```

An empty pool means the data file is missing entries for that slot, and the solver could never produce a valid plan. Failing fast here is much better than a confusing INFEASIBLE later.

■ Key Takeaway

Pipeline summary: Excel -> ColumnMapper (canonical schema) -> DataCleanser (no junk rows) -> PoolBuilder (one DataFrame per base slot, expanded if needed). The solver receives a Python dict **{slot_name: DataFrame}** and is then free to pick candidates without worrying about messy data.

Part D — The Solver Deep Dive

Part A introduced boolean switches and equations. Now we open the hood of **MenuSolver** and see exactly how the model is built. Take this part slowly — every other part references something here.

Chapter 14 — Cell Architecture

The solver thinks in *cells*, not just slots. A cell is a single (day, slot, list-of-candidates) triple. For a five-day plan with eleven slots, there are 55 cells. Each cell ends up with its own group of boolean switches plus an "exactly one" constraint.

The `_Cell` class

Defined at the top of `src/solver/menu_solver.py`:

```
class _Cell:
    __slots__ = (
        'd_idx', 'date', 'slot_id', 'base_slot',
        'cand_df', 'theme_pref_flags', 'x_vars', 'cand_rows',
    )
    def __init__(self, d_idx, date, slot_id, base_slot,
                  cand_df, theme_pref_flags):
        self.d_idx = d_idx # day index (0..N-1)
        self.date = date # actual date object
        self.slot_id = slot_id # 'veg_dry__1' (expanded)
        self.base_slot = base_slot # 'veg_dry' (base)
        self.cand_df = cand_df # filtered DataFrame
        self.theme_pref_flags = ... # which candidates match theme
        self.x_vars = [] # filled in later by build_decision_variables
        self.cand_rows = [] # parallel list to x_vars
```

Why `__slots__`? It tells Python "these are all the attributes I will ever have", which makes the object smaller and slightly faster — important when you have hundreds of cells. Not magical; just tidy.

Cells vs slots vs base slots — three names that look similar

Term	Example	Meaning
base slot	veg_dry	The category of dish.
slot_id	veg_dry__1	An expanded instance — one per item that day.
cell	(day=Mon, slot_id=veg_dry__1)	One decision point in the planner.

Walking through cell creation

MenuSolver._build_cells() loops over every (day, expanded slot) pair and builds a Cell for each that isn't in **skip_cells**:

```
for di, d in enumerate(dates):
    for slot_id in expanded_slots:
        base = _base_slot(slot_id)
        if (d, base) in self.skip_cells:
            continue # custom skip rule

        # Phase 1 filters were applied earlier into a per-(day, slot)
        # cache. Pull the result out:
        pool, pref_mask, day_type = cache[di, slot_id]
        if len(pool) == 0:
            raise RuntimeError(...) # empty pool = infeasible

        # Sample to a manageable size (Chapter 15)
        sampled, theme_flags = _sample_cell_candidates(...)
        cells.append(_Cell(di, d, slot_id, base, sampled, theme_flags))
```

■ Key Takeaway

Cell = (day, expanded slot, list of candidates). Everything downstream — boolean variables, hard constraints, objective terms — operates on Cells. If you ever need to debug “why didn't slot X get item Y?”, find the Cell and inspect its cand_df.

Chapter 15 — Candidate Sampling with Priority

There are 530 dishes total. After pre-filtering, a single (day, slot) might still have 200+ valid candidates. Creating 200 boolean switches for every cell would explode the model — 5 days x 11 slots x 200 = 11,000 switches. That is too many.

We sample a smaller subset of candidates per cell. The default cap varies by slot:

```
DEFAULT_CAP_BY_SLOT = {
    'rice': 1600, 'healthy_rice': 1200, 'veg_gravy': 1400,
    'nonveg_main': 1400, 'curd_side': 1400, 'veg_dry': 1100,
    'bread': 1100, 'starter': 1200, 'soup': 900, 'salad': 900,
    'dal': 1000, 'dessert': 1000, 'welcome_drink': 1000,
    'sambar': 900, 'rasam': 900,
}
DEFAULT_CAP = 900
```

Wait — those numbers look big! That's because they're already intentionally bigger than the largest pool, so by default we keep everything. The cap matters only when multi-restart (Chapter 18) shrinks the pool to force diversity.

Priority sampling — keep theme matches

If we are sampling Tuesday's veg_dry slot and there are 5 Chinese-style items in the pool, we want all 5 to survive sampling. Random sampling could throw them out and leave us with no theme-matching dishes. **`_sample_with_priority`** ensures priority items always make it in:

```
def _sample_with_priority(pool, cap, priority_mask, rng):
    if len(pool) <= cap:
        return pool # everything fits
    pri, oth = pool[priority_mask], pool[~priority_mask]
    if len(pri) >= cap:
        return pri.sample(cap, ...) # plenty of priority
    if len(pri) == 0:
        return pool.sample(cap, ...) # no priority, sample uniformly
    need = cap - len(pri)
    if len(oth) > need:
        oth = oth.sample(need, ...)
    return pd.concat([pri, oth]) # priority + topped up
```

■ Analogy

Imagine packing a lunchbox. You only have room for 10 items. There are 30 in the fridge but the recipe specifically says “make sure all the spicy ones go in”. So first you grab every spicy item (say there are 4), then you randomly fill the remaining 6 slots from the rest.

`_sample_with_priority` does exactly this for the solver.

Chapter 16 — Building the CP-SAT Model Step by Step

Now the cells exist with their candidate DataFrames. `_build_decision_variables()` walks every cell and creates the boolean switches plus the bookkeeping needed by every rule downstream.

The variable-creation loop

```
for cell in cells:
    di, slot_id, base = cell.d_idx, cell.slot_id, cell.base_slot
    x_vars, cand_rows = [], []

    for j in range(len(cell.cand_df)):
        row = cell.cand_df.iloc[j]
        item_base = _norm_str(row.get('item', ''))

        # 1. Create the boolean switch
        var = model.NewBoolVar(f'x_d{di}_{slot_id}_{j}')
        x_vars.append(var)
        cand_rows.append(row)

        # 2. Track for "no repeats" rule
        item_to_vars.setdefault(item_base, []).append(var)

        # 3. Track for premium rule
        if cfg.premium_flag_col and int(row.get(cfg.premium_flag_col, 0)) == 1:
            day_premium_vars.setdefault(di, []).append(var)

        # 4. Track for color rules
        if base in cfg.color_slots:
            col = _norm_color(row.get(cfg.color_col, 'unknown'))
            if col != 'unknown':
                day_color_vars.setdefault((di, col), []).append(var)
                if base == 'rice':
                    day_rice_color_vars.setdefault((di, col), []).append(var)
                elif base == 'veg_gravy':
                    day_gravy_color_vars.setdefault((di, col), []).append(var)

        # 5. Track for welcome drink color rule
        if base == 'welcome_drink':
            col = _norm_color(row.get(cfg.color_col, 'unknown'))
            if col != 'unknown':
                day_welcome_color_vars.setdefault((di, col), []).append(var)

        # 6. Track for monday-mix rule
        if day_types[di] == 'mix' and base not in EXEMPT_FROM_CUISINE:
            cf = _norm_str(row.get(cfg.cuisine_col, ''))
            if cf == cfg.cuisine_south_value:
                monday_south_lits.append(var)
            elif cf == cfg.cuisine_north_value:
                monday_north_lits.append(var)

    # 7. The exactly-one constraint for this cell
    model.Add(sum(x_vars) == 1)
```



```
cell.x_vars = x_vars
cell.cand_rows = cand_rows
```

That single loop does seven things at once because they all need access to the same row data. Every dictionary it builds (**item_to_vars**, **day_color_vars**, etc.) becomes part of the **SolverContext** passed to every rule.

Built-in color constraints

Some rules are so universal they live directly in the solver instead of as separate rule classes. Color constraints are the main example:

```
def _add_color_constraints(self, model, dates, day_types, known_colors,
                          day_color_vars, day_rice_color_vars,
                          day_gravy_color_vars):
    for di, _ in enumerate(dates):
        day_type = day_types[di]
        min_dist = _min_distinct_for_day(cfg, day_type)

        # Rule: at most N items of the same color per day
        for col in known_colors:
            lits = day_color_vars.get((di, col), [])
            if lits:
                model.Add(sum(lits) <= cfg.max_same_color_per_day)

        # Rule: at least M distinct colors per day
        y_vars = []
        for col in known_colors:
            lits = day_color_vars.get((di, col), [])
            if not lits:
                continue
            y = model.NewBoolVar(f'y_color_{di}_{col}')
            _link_any(model, lits, y) # y = 1 iff any lit is on
            y_vars.append(y)
        if y_vars:
            model.Add(sum(y_vars) >= min_dist)

        # Rule: rice and gravy colors must differ (skipped on Chinese days)
        if not (chinese_skip):
            for col in known_colors:
                r = day_rice_color_vars.get((di, col), [])
                g = day_gravy_color_vars.get((di, col), [])
                if r and g:
                    model.Add(sum(r) + sum(g) <= 1)
```

Applying user-defined rules

After built-in constraints, the solver iterates every loaded rule and calls **rule.apply(model, ..., context)**. Hard rules raise on failure (causing the solve to fail); soft rules log + record + continue.

```
for rule in self.menu_rules:
    try:
        rule.apply(model, {}, None, context)
    except Exception as e:
        severity = getattr(rule, 'severity', MenuRuleSeverity.HARD)
        if severity == MenuRuleSeverity.HARD:
            raise RuntimeError(f"Hard menu rule '{rule.name}' failed: ...")
        # SOFT: record + continue
        self._record_rule_failure(rule, 'apply', e)
```

■ Key Takeaway

Model-building is a sweep that does seven types of bookkeeping plus the universal “exactly one” constraint per cell, then asks every rule to add its own constraints. The heavy lifting is in the loop you just read.

Chapter 17 — The Objective Function

Hard constraints alone tell the solver “this combination is okay or not okay”. They do NOT tell it which valid combination is best. The objective function is how we express preferences.

How the objective is assembled

```
def _build_objective(self, model, cells, rng, similarity, context):
    obj_terms = []

    if similarity:
        # Regeneration mode: prefer items that resemble what the user
        # already liked (positive scores) and avoid the items they asked
        # to replace (negative scores)
        for cell in cells:
            for var, row in zip(cell.x_vars, cell.cand_rows):
                sc = int(similarity.get(
                    (cell.date, cell.slot_id, _norm_str(row.get('item', ''))), 0
                ))
                if sc:
                    obj_terms.append(var * sc)
        # Plus a tiny random tiebreaker so two equally-good plans differ run-to-run
        for cell in cells:
            for var in cell.x_vars:
                obj_terms.append(var * rng.randint(0, 3))
    else:
        # Fresh planning: pure random scoring drives variety
        for cell in cells:
            for var in cell.x_vars:
                obj_terms.append(var * rng.randint(0, 1000))

    # Plus contributions from soft rules (theme bonuses, fallback penalties...)
    for rule in self.menu_rules:
        try:
            terms = rule.get_objective_terms(model, context)
            obj_terms.extend(terms)
        except Exception as e:
            self._record_rule_failure(rule, 'get_objective_terms', e)

    if obj_terms:
        model.Maximize(sum(obj_terms))
```

Three sources of objective terms

Source	Purpose
Random per-variable scores	Tie-breaker. Without this, multiple equally-valid plans would always come out the same; with it, two runs produce slightly different results.
Similarity scores (regen only)	Positive points for items resembling the original; negative points for explicitly-replaced items.
Soft rule contributions	Big bonuses (+1M for theme starter), big penalties (-2M for non-theme fallback).

Why such giant numbers?

The random tie-breaker is in the 0-1000 range. The theme bonus is 1,000,000. This intentional ratio means: **theme preferences ALWAYS dominate randomness**. Soft rules pull the solver hard in their direction. Randomness only matters when soft rules are equal.

■ Tip

When tuning a soft rule, think about its weight relative to other soft rules and to the random score range. A weight of 100 will be drowned out by randomness and look broken; a weight of 10,000,000 will overshadow every other soft rule. The codebase uses 1,000,000 to 2,000,000 as the comfortable middle ground.

Chapter 18 — Multi-Restart Strategy

Sometimes the solver tries hard but cannot find a feasible solution. Maybe the candidate sample was unlucky — too many similar items got kept and the constraint set is over-determined. The solver answers INFEASIBLE.

We do not give up on the first attempt. **MenuSolver.solve()** implements a multi-restart strategy:

```
cap_multipliers = (1, 2) # try normal pool, then 2x bigger
restarts_per_mult = 4 # 4 different seeds per multiplier

for mult in cap_multipliers:
    cap_default = self.cfg.cap_default * mult
    cap_by_slot = {k: v * mult for k, v in self.cfg.cap_by_slot.items()}

    for r in range(restarts_per_mult):
        attempt_seed = base_seed + mult * 1000 + r * 17
        rng = random.Random(attempt_seed)
        self.cfg.seed = attempt_seed

        try:
            cells = self._build_cells(...)
            chosen = self._solve_cpsat(...)
            return self._rows_to_week_plan(chosen, ...) # success!
        except RuntimeError as e:
            last_err = e
            continue # try again
```

That gives 8 attempts in total:

Attempt	Seed	Pool size	Outcome
1	7	1x	fail
2	24	1x	fail
3	41	1x	fail
4	58	1x	fail
5	1007	2x	fail
6	1024	2x	success!
7	1041	2x	(skipped)
8	1058	2x	(skipped)

■ Analogy

Like trying different keys until one fits the lock. Each restart is a different random sampling of candidates and a different starting point for the CP-SAT search.

Per-attempt failure tracking

Each attempt clears its own **rule_failures** list at the start. Only the failures from the *winning* attempt make it into the response — failures from abandoned attempts would be misleading.

```
self._current_attempt_seed = attempt_seed
self.rule_failures = [] # fresh for this attempt
```

■ Key Takeaway

Multi-restart trades a little compute for a lot of robustness. The user almost never sees an INFEASIBLE because the solver tries 8 different combinations behind the scenes. If all 8 fail, something is genuinely wrong with the rule config or the data.

Chapter 19 — Solution Extraction

Once the solver finishes, we have the **solver** object plus all the boolean switches. To turn that into a usable plan, we read which switch is on for each cell:

```
def _extract_solution_rows(self, solver, cells, dates):
    chosen = {d: {} for d in dates}
    for cell in cells:
        pick_idx = next(
            (j for j, var in enumerate(cell.x_vars) if solver.Value(var) == 1),
            None,
        )
        if pick_idx is None:
            raise RuntimeError('Solver solution missing selection in a cell.')
        chosen[cell.date][cell.slot_id] = cell.cand_rows[pick_idx]
    return chosen
```

chosen is now a dict mapping date -> {slot_id -> pandas row}. One last step formats it for output:

```
def _rows_to_week_plan(self, chosen_rows, dates, expanded_slots):
    week_plan = {}
    for d in dates:
        day_out = {}
        for slot_id in expanded_slots:
            if slot_id in chosen_rows[d]:
                day_out[slot_id] = _fmt_item_with_color(
                    chosen_rows[d][slot_id], self.cfg.color_col
                )
        day_out.update(CONSTANT_ITEMS) # white_rice, papad, pickle, chutney
        week_plan[d] = day_out
    return week_plan
```

_fmt_item_with_color turns a row into the display string used throughout the system:

“jeera_rice(Y)” — item name plus a single-letter color suffix in parentheses.

CONSTANT_ITEMS appends white rice, papad, pickle, and chutney unconditionally — they're served every day so the solver doesn't waste switches on them.

■ Key Takeaway

Five steps to a finished plan: cells -> variables -> constraints -> solve -> read switches.

Multi-restart wraps the whole thing. **SolutionFormatter** in solver/solution_formatter.py later turns this dict into the JSON the API returns.

Part E — All Rules as Equations

This is the heart of the book. We will look at every rule in the system and show exactly how it becomes math the solver understands. Then we will walk through one complete menu generation from start to finish.

Chapter 20 — The Rule Translation Master Class

Every rule belongs to one of three categories based on which phase it operates in. This single table is the entire rulebook in one page:

Rule	Phase	Equation / Action
item_cooldown	Pre-filter	$\text{pool} = \text{pool}[\sim \text{pool.item.isin}(\text{banned_recently})]$
ricebread_gap	Pre-filter	if banned today: $\text{pool} = \text{pool}[\text{pool.is_rice_bread} == 0]$
theme_slot_filter	Pre-filter	$\text{pool} = \text{pool}[\text{pool.is_chinese_fried_rice} == 1]$ (etc.)
nonveg_biryani_weekly	Pre-filter + CP-SAT	$\text{Sum}(\text{biryani_days}) \leq \text{max_per_week}$
nonveg_dry_preference	Pre-filter	if slot_num ≥ 2 : $\text{pool} = \text{pool}[\text{is_nonveg_dry} == 1]$
unique_items	CP-SAT (hard)	For each item: $\text{Sum}(\text{uses across days}) \leq 1$
premium	CP-SAT (hard)	$\text{Sum}(\text{daily}) \leq \text{max_per_day}$; $\text{Sum}(\text{week})$ in [min, max]
color_variety	CP-SAT (hard)	$\text{Sum}(\text{distinct color flags per day}) \geq \text{min_distinct}$
color_pairing	CP-SAT (hard)	For each color: $\text{Sum}(\text{red_in_A}) + \text{Sum}(\text{red_in_B}) \leq 1$
coupling	CP-SAT (hard)	$\text{bread_ricebread} \Rightarrow \text{rice_liquid}$; $\text{deepfried_starter pair}$
curd_side	CP-SAT (hard)	$\text{biryani_day OR pulao_day} \Rightarrow \text{raita_chosen}$
theme_day	CP-SAT (hard)	Mix day: $\text{Sum}(\text{south}) \geq 1$ AND $\text{Sum}(\text{north}) \geq 1$
welcome_drink_color	CP-SAT (hard)	For each color c: $\text{Sum}(c_today) + \text{Sum}(c_tomorrow) \leq 1$
cuisine	CP-SAT (hard)	On configured days: $\text{Sum}(\text{cuisine_X items}) \geq 1$
theme_starter_pref	Soft (objective)	objective $+= 1,000,000 * \text{Sum}(\text{theme_starter_chosen})$
theme_fallback_penalty	Soft (objective)	objective $-= 2,000,000 * \text{Sum}(\text{non_theme_chosen})$
week_signature_cooldown	Soft (CP-SAT)	For each recent sig: $\text{Sum}(\text{matching}) \leq N - 1$
ingredient_ban (custom)	Pre-filter	$\text{pool} = \text{pool}[\sim \text{pool.key_ingredient.isin}(\text{banned})]$
item_frequency (custom)	CP-SAT (hard)	$\text{min_per_week} \leq \text{Sum}(\text{matching_days}) \leq \text{max_per_week}$
slot_day_restriction (custom)	Skip-cells	$\text{skip_cells.add}((\text{date}, \text{slot}))$ for disallowed weekdays

■ Key Takeaway

Every rule is one of: (1) filter the candidate list, (2) add a math equation to the model, (3) add a bonus/penalty to the objective, or (4) skip cell creation entirely. Three categories — that's it.

The three phases, side by side

Let me show you the same logical rule expressed in each phase so you see the trade-offs.

Pre-filter — fast, no math, but coarse:

```
def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
    # Tuesday must be Chinese: keep only is_chinese_fried_rice == 1
    if day_type == 'chinese' and base_slot == 'rice':
        pool = pool[pool['is_chinese_fried_rice'] == 1]
    return pool
```

Result: zero non-Chinese rice switches ever get created on Tuesday. The solver doesn't even know they exist.

CP-SAT hard — math, can express complex relationships:

```
def apply(self, model, variables, menu_data, context):
    # At most one premium item per day
    for di, lits in day_premium_vars.items():
        if lits:
            model.Add(sum(lits) <= self.max_per_day)
```

Result: the solver creates premium switches and at solve time enforces the inequality. More expensive than pre-filter, but it can talk about relationships among different cells.

Soft objective — preference, never blocks:

```
def get_objective_terms(self, model, context):
    ok_vars = []
    for di in range(len(dates)):
        for scell in find_cells(cells, di, 'starter'):
            theme_lits = [v for v, pref in zip(scell.x_vars,
                                                scell.theme_pref_flags) if pref]

            if theme_lits:
                ok = model.NewBoolVar(f'starter_theme_ok_{di}')
                link_any(model, theme_lits, ok)
                ok_vars.append(ok)

    if ok_vars:
        return [sum(ok_vars) * self.bonus_weight]
    return []
```

Result: every starter that matches the theme adds 1,000,000 to the score. The solver tries to maximize total score, so it will choose theme starters when possible — but if it can't, the plan still succeeds with a lower score.

■ **Tip**

When designing a new rule, ask: “Can I express this by removing items from a list?” If yes, use pre-filter. “Does it relate items across different cells?” Use CP-SAT hard. “Is this just a preference that should still allow a fallback?” Use objective.

Chapter 21 — End-to-End: How One Menu Gets Generated

Time to tie everything together. We will follow one real request through the entire system. Client = Tekion (which has 3 custom rules). Start date = Monday May 4, 2026. Days = 5.

Step 1 — The Streamlit click

User selects Tekion in the dropdown, picks May 4 as start date, moves the slider to 5 days, and clicks "Generate Menu Plan". Streamlit calls:

```
client.plan(
    client_name='Tekion',
    start_date='2026-05-04',
    num_days=5,
    time_limit_seconds=180,
)
# Internally: POST http://localhost:5000/api/v1/plan
# headers: Authorization: Bearer eyJ0eXAi...
```

Step 2 — Flask receives the request

Three decorators run before the handler body:

```
@app.route('/api/v1/plan', methods=['POST'])
@require_api_auth() # 1. Bearer token verified -> g.api_user populated
@rate_limit("plan") # 2. Token bucket: 10/min, burst 10
@solver_gate # 3. Queue if 2 already running; max 8 queued
def plan_menu():
    ...
```

Auth passes (valid token). Rate limit passes (this user has used 0 plans this minute). Solver gate passes (no other solves running). We enter the handler.

Step 3 — Loading the client config

The handler calls `_prepare_solver_inputs(data)`, which first validates the client name and then loads the config from Supabase:

```
client_name = 'Tekion'
client_cfg = ClientConfigLoader().get_client('Tekion')

# What we get back:
ClientConfig(
    name='Tekion',
    active_slots=['bread', 'veg_dry', 'rice', 'veg_gravy',
                  'nonveg_main', 'dal', 'sambar', 'rasam',
```

```

        'curd_side', 'dessert', 'salad'],
    slot_counts={'bread': 1, 'veg_dry': 1, 'rice': 1, ...},
    theme_map={'monday': 'mix', 'tuesday': 'chinese',
               'wednesday': 'biryani', 'thursday': 'south',
               'friday': 'north'},
)

```

Step 4 — Loading the menu data

Cached on first call, instant on subsequent calls:

```

df, pools = _get_menu_data()

# pools is a dict:
# {
#   'rice': <DataFrame: 47 rows>,
#   'bread': <DataFrame: 28 rows>,
#   'veg_dry': <DataFrame: 92 rows>,
#   'veg_gravy': <DataFrame: 71 rows>,
#   'nonveg_main': <DataFrame: 38 rows>,
#   'dal': <DataFrame: 12 rows>,
#   ... etc
# }
# Total: ~530 dishes split across 15 base slots

```

Step 5 — Loading rules (generic + Tekion-specific)

```

rules, skip_cells = _rules_and_skip_for_client('Tekion', weekday_dates)

# rules now contains:
# [Generic rules (14 of them):]
# ItemCooldownMenuRule(cooldown_days=20)
# RiceBreadGapMenuRule(gap_days=10)
# ThemeSlotFilterRule(...)
# UniqueItemsMenuRule()
# ThemeDayMenuRule()
# CouplingMenuRule()
# CurdSideMenuRule()
# PremiumMenuRule(max_per_day=1, min=1, max=2)
# WelcomeDrinkColorMenuRule()
# WeekSignatureCooldownMenuRule(cooldown_days=30)
# ThemeStarterPreferenceRule(bonus=1_000_000)
# ThemeFallbackPenaltyRule(penalty=2_000_000)
# NonvegDryPreferenceRule()
# NonvegBiryaniWeeklyRule(max_per_week=1)
# [Tekion-specific (3 of them):]
# IngredientBanRule(banned={'mushroom'})
# ItemFrequencyRule(rice + is_liquid_rice, min=1, max=1)
# SlotDayRestrictionRule(nonveg_main, allowed=[Mon,Wed,Fri])

```

```
# skip_cells is computed by SlotDayRestrictionRule:
# {
# (Tue 2026-05-05, 'nonveg_main'),
# (Thu 2026-05-07, 'nonveg_main'),
# }
```

Step 6 — Loading history from Supabase

```
banned, rb_ban, recent_sigs = _build_history_context(
    df, 'Tekion', start_date=2026-05-04,
    weekday_dates=[Mon, Tue, Wed, Thu, Fri],
    window_days=45,
)

# banned is a dict mapping each date to items recently used:
# {
# Mon 2026-05-04: {'jeera_rice', 'paneer_butter_masala', 'aloo_gobi', ...},
# Tue 2026-05-05: {... ~67 items ...},
# Wed 2026-05-06: {...},
# Thu 2026-05-07: {...},
# Fri 2026-05-08: {...},
# }

# rb_ban: {date: True/False}
# True means rice-bread is banned today because it was used in last 10 days

# recent_sigs: a set of week signatures used in the last 30 days
```

Step 7 — Pre-filtering pools, day by day

Now we walk every (day, slot) combination and apply pre-filters:

```
# Example: Tuesday, base_slot='rice', day_type='chinese'
# Original rice pool: 47 items
# Apply filters in order:

# 1. ItemCooldownMenuRule.pre_filter_pool
# Remove banned items (recent 20 days)
# 47 -> 38 items

# 2. ThemeSlotFilterRule.pre_filter_pool
# Tuesday is Chinese: keep only is_chinese_fried_rice == 1
# 38 -> 4 items (Veg Fried Rice, Schezwan Rice, Burnt Garlic Rice, ...)

# 3. RiceBreadGapMenuRule.pre_filter_pool
# base_slot != 'bread', no-op

# 4. NonvegDryPreferenceRule.pre_filter_pool
# base_slot != 'nonveg_main', no-op
```

```
# 5. IngredientBanRule.pre_filter_pool (Tekion's custom rule)
# Remove items where key_ingredient == 'mushroom'
# 4 -> 4 items (none of the Chinese rices use mushroom)

# Final: 4 candidates for Tuesday Rice
```

We do this for every (day, slot) cell. Skip the cells in **skip_cells** entirely (no nonveg_main on Tuesday or Thursday).

Step 8 — Building cells and switches

```
# After pre-filtering, we have ~52 cells (5 days x 11 slots minus 2 skipped).
# Each cell has a candidate count between 4 and ~50.
# Total switches across all cells: roughly 1,800.

# For each cell, MenuSolver creates the boolean switches:
for cell in cells:
    for j, row in enumerate(cell.cand_df):
        var = model.NewBoolVar(f'x_d{cell.d_idx}_{cell.slot_id}_{j}')
        cell.x_vars.append(var)

# And the exactly-one constraint:
model.Add(sum(cell.x_vars) == 1)
```

Step 9 — Adding hard constraints (the equations)

Each rule.apply() contributes its equations. A small sample:

```
# UniqueItemsMenuRule:
# For 'jeera_rice' that appears in cells [Mon-rice, Wed-rice]:
model.Add(Mon_rice_jeera + Wed_rice_jeera <= 1)
# ... repeat for every item base across all its cells

# PremiumMenuRule (max_per_day=1):
model.Add(Mon_paneer_butter + Mon_paneer_tikka + Mon_paneer_kebab <= 1)
# ... repeat for each day
# PremiumMenuRule (week range [1, 2]):
model.Add(Mon_premium + Tue_premium + Wed_premium + Thu_premium + Fri_premium >= 1)
model.Add(Mon_premium + Tue_premium + Wed_premium + Thu_premium + Fri_premium <= 2)

# ThemeDayMenuRule (Monday is mix day):
model.Add(Mon_south_lit_1 + Mon_south_lit_2 + ... >= 1) # at least 1 south
model.Add(Mon_north_lit_1 + Mon_north_lit_2 + ... >= 1) # at least 1 north

# ItemFrequencyRule (Tekion: liquid rice exactly once per week):
model.Add(Mon_liquid + Tue_liquid + Wed_liquid + Thu_liquid + Fri_liquid >= 1)
model.Add(Mon_liquid + Tue_liquid + Wed_liquid + Thu_liquid + Fri_liquid <= 1)

# Color variety (>= 4 distinct colors per day):
model.Add(y_red + y_green + y_yellow + y_white + y_brown >= 4) # for each day
```



```
# ... and roughly 20 more equations from the other rules
```

Step 10 — Adding soft objective terms

```
obj_terms = []

# Random tiebreaker (each var weighted 0..1000):
obj_terms.append(Mon_rice_jeera * 472)
obj_terms.append(Mon_rice_lemon * 891)
# ... for every var

# Theme starter preference (+1M per match):
obj_terms.append((Mon_starter_south_indian_match) * 1_000_000)
obj_terms.append((Tue_starter_chinese_match) * 1_000_000)
# ... for every day's starter

# Theme fallback penalty (-2M per non-match):
obj_terms.append(theme_fallback_bool * -2_000_000)

# Week signature cooldown (penalty if today's plan matches a recent week)
# ...

model.Maximize(sum(obj_terms))
```

Step 11 — CP-SAT solves

```
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 180
solver.parameters.random_seed = 7
solver.parameters.num_search_workers = 9 # depends on concurrency

status = solver.Solve(model)
# status == FEASIBLE (found a valid plan with score ~5.4M)
```

Internally the CP-SAT engine uses techniques like SAT-style clause propagation, conflict-driven learning, and branch-and-bound. We don't have to understand any of that — we just trust it works.

Step 12 — Reading switches back

```
chosen = {}
for cell in cells:
    for j, var in enumerate(cell.x_vars):
        if solver.Value(var) == 1:
            chosen[cell.date][cell.slot_id] = cell.cand_rows[j]
            break
```

```
# chosen is now:
# {
# Mon 2026-05-04: {
# 'rice': <row: jeera_rice>,
# 'bread': <row: tandoori_naam>,
# 'veg_dry': <row: aloo_gobi>,
# ...
# },
# Tue 2026-05-05: {
# 'rice': <row: veg_fried_rice>,
# 'bread': <row: garlic_butter_naam>,
# ... (no nonveg_main on Tuesday)
# },
# ... etc
# }
```

Step 13 — Formatting and returning

```
formatter = SolutionFormatter(week_plan, plan_dates, theme_map=...)
response_body = {
    'success': True,
    'message': 'Menu plan generated for Tekion',
    'solution': formatter.to_dict(),
}

# response_body['solution'] looks like:
{
    '2026-05-04': {
        'theme': 'Mix of South + North',
        'day_type': 'mix',
        'items': {
            'rice': {
                'display_name': 'Flavor Rice',
                'item': 'jeera_rice(Y)',
                'item_base': 'jeera_rice',
            },
            'bread': {...},
            ...
        }
    },
    '2026-05-05': {...},
    ...
}
```

Step 14 — Streamlit renders the table

The Flask response goes back over HTTP. Streamlit's `MenuApiClient` parses it, fills `st.session_state.plan`, and `st.rerun()` triggers a fresh render. The HTML is built by the formatters in `ui/formatters.py` using `format_item_html` to produce the colored pills.

■ Key Takeaway

That is the entire flow — from click to colored table — in 14 steps. Every other part of this book zooms in on one of these steps. If you ever feel lost, come back to this chapter and find which step you're trying to understand.

Part F — Individual Rule Deep Dives

Part E gave you the master class. Now we zoom in on each rule with examples, edge cases, and the “why” behind each decision. You can read these in any order — pick the rule you’re curious about and skip ahead.

Chapter 22 — Cooldown Rules

Three rules in **cooldown_rules.py** together prevent boredom: item cooldown, rice-bread gap, and week signature cooldown.

ItemCooldownMenuRule

“Don’t serve the same item within 20 days.” This is the simplest and most important rule. It is pure pre-filter:

```
def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
    banned_by_date = ctx.get('banned_by_date', {})
    banned = banned_by_date.get(date, set())
    if banned and len(pool) > 0:
        pool = pool[~pool['item'].isin(banned)]
    return pool
```

Where does **banned_by_date** come from? **HistoryManager** queries Supabase’s **menu_history** table for the last 20 days and builds a per-date set of recently-used items. Const slots like **white_rice** are excluded (they repeat by design).

RiceBreadGapMenuRule

“Wait at least 10 days between rice-based breads.” Rice-based breads (dosa, neer dosa, etc.) are heavy and similar in texture. Serving them too close together feels repetitive.

```
def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
    if base_slot != 'bread':
        return pool # only applies to bread
    ricebread_ban_day = ctx.get('ricebread_ban_day', {})
    if ricebread_ban_day.get(date, False):
        if 'is_rice_bread' in pool.columns:
            pool = pool[pool['is_rice_bread'] == 0]
    return pool
```

ricebread_ban_day is a per-date boolean: True if any rice-bread was served in the last 10 days. If True, we drop all rice-bread candidates from today's bread slot.

WeekSignatureCooldownMenuRule

"Don't replay last month's exact menu." This one is interesting — it operates at week granularity, not item. The system stores a deterministic hash (signature) of every saved week. If the candidate switches today exactly match a recent signature, we forbid that specific combination.

```
def apply(self, model, variables, menu_data, context):
    cells = context.get('cells', [])
    recent_sigs = context.get('recent_sigs', set())

    for sig in recent_sigs:
        exp = parse_signature_to_expected_map(sig)
        # exp = {'2026-04-15', 'rice': 'jeera_rice',
        # ('2026-04-15', 'bread'): 'naan', ...}
        lits = []
        for cell in cells:
            want = exp.get((cell.date.isoformat(), cell.slot_id))
            if not want: lits = []; break
            found = next((v for v, r in zip(cell.x_vars, cell.cand_rows)
                          if r['item'] == want), None)
            if found is None: lits = []; break
            lits.append(found)
        if lits and len(lits) >= 2:
            model.Add(sum(lits) <= len(lits) - 1)
            # Translation: at least one position must differ
```

This rule is SOFT — it can fail without crashing the solve. The constraint "at least one position must differ" is a clever way to forbid an exact replay while allowing nearly-identical plans.

Chapter 23 — Theme Rules

Four rules in **theme_rules.py** together implement the weekday-to-cuisine theme system: Tuesday is Chinese, Wednesday is Biryani, etc.

ThemeSlotFilterRule (pre-filter)

This is the heavy lifter. Different filters per day type:

```
def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
    if day_type == 'chinese': return self._filter_chinese(pool, base_slot, cfg)
    if day_type == 'biryani': return self._filter_biryani(pool, base_slot, cfg)
    if day_type in ('south', 'north'):
        return self._filter_cuisine(pool, base_slot, day_type, cfg)
    return pool # mix and holiday days: no theme filtering
```

Chinese day uses flag columns to filter rice, veg gravy, and nonveg_main. Veg dry uses text heuristics (looking for keywords like "manchurian", "schezwan", etc.). South/north days filter by **cuisine_family** column.

ThemeDayMenuRule (hard CP-SAT)

Monday is "mix" day — a Monday menu must include at least one south-Indian item AND at least one north-Indian item. This needs math because filtering can't enforce "at least one of each":

```
def apply(self, model, variables, menu_data, context):
    if any(dt_ == 'mix' for dt_ in day_types):
        if south_lits:
            model.Add(sum(south_lits) >= 1)
        if north_lits:
            model.Add(sum(north_lits) >= 1)
```

ThemeStarterPreferenceRule (soft objective)

Even on Monday's mix day, we'd love the starter to match the theme. But starters are sparse — sometimes there isn't one that fits. So this is a soft preference: bonus 1,000,000 per starter that matches:

```
def get_objective_terms(self, model, context):
    ok_vars = []
    for di in range(len(dates)):
        for scell in find_cells(cells, di, 'starter'):
            theme_lits = [v for v, p in zip(scell.x_vars,
                                           scell.theme_pref_flags) if p]
            if theme_lits:
```

```
ok = model.NewBoolVar(...)
link_any(model, theme_lits, ok)
ok_vars.append(ok)
return [sum(ok_vars) * self.bonus_weight] if ok_vars else []
```

ThemeFallbackPenaltyRule (soft objective)

If a non-theme item slips into a slot that should be themed, we subtract 2,000,000 from the score. Combined with the theme starter bonus, this strongly steers the solver toward theme compliance without making it impossible when no theme item fits.

■ Key Takeaway

Theme system = pre-filter (eliminate non-theme items where possible) + hard mix-day requirement + soft bonus for theme starters + soft penalty for fallbacks. Four rules working together to implement what feels like one feature.

Chapter 24 — Color Rules

Visual variety matters. The system enforces three color rules: minimum distinct colors per day, maximum same-color count per day, and welcome drink color non-repetition across consecutive days.

Color variety (minimum distinct)

“At least 4 distinct colors per day.” The trick is that “distinct” isn’t natively expressible in CP-SAT — we have to encode it via indicator variables:

```
# For each day, for each known color, create a y variable:
# y_color_di_red = 1 iff at least one selected switch is red

for di in range(len(dates)):
    y_vars = []
    for col in known_colors:
        lits = day_color_vars.get((di, col), [])
        if lits:
            y = model.NewBoolVar(f'y_color_{di}_{col}')
            link_any(model, lits, y) # y = OR of lits
            y_vars.append(y)
    model.Add(sum(y_vars) >= 4) # at least 4 colors present
```

Maximum same-color

“At most 2 items of the same color per day.” This one is direct:

```
for col in known_colors:
    lits = day_color_vars.get((di, col), [])
    if lits:
        model.Add(sum(lits) <= cfg.max_same_color_per_day)
```

Color pairing (rice vs veg gravy)

“Rice and veg gravy on the same day shouldn’t share a color.” Picture jeera rice (yellow) with dal makhani (red) — distinct colors. Now picture both yellow — visually monotonous. The rule:

```
for col in known_colors:
    rice_red = day_rice_color_vars.get((di, col), [])
    gravy_red = day_gravy_color_vars.get((di, col), [])
    if rice_red and gravy_red:
        model.Add(sum(rice_red) + sum(gravy_red) <= 1)
        # i.e. cannot pick both a red rice and a red gravy
```

Welcome drink color (consecutive days)

"Yesterday's welcome drink color shouldn't be today's color." Implemented as a CP-SAT pairwise constraint:

```
for di in range(len(dates) - 1):
    for col in known_welcome_colors:
        a = day_welcome_color_vars.get((di, col), [])
        b = day_welcome_color_vars.get((di + 1, col), [])
        if a and b:
            model.Add(sum(a) + sum(b) <= 1)
```

■ Key Takeaway

Color rules use the same trick as Chinese day filtering: track which switches contribute to each color via a dictionary (`day_color_vars`) populated during variable creation. This bookkeeping is why `_build_decision_variables` does seven things at once.

Chapter 25 — The Coupling Rule (the clever one)

This is the most intricate rule in the codebase. It says: "If you serve a rice-based bread, you must also serve a liquid rice and a deep-fried starter, and a deep-fried veg dry must co-occur with a rice-bread + liquid-rice day. Also limit to one rice-bread day and one deep-fried-veg-dry day per week."

Translated into CP-SAT logic:

```
for di in range(len(dates)):
    # Detect what's in each cell type via OR (link_any)
    bread_rb = "any rice-bread chosen today?" (bool var)
    rice_liq = "any liquid rice chosen today?" (bool var)
    starter_df = "any deep-fried starter chosen today?" (bool var)
    vegdry_df = "any deep-fried veg dry chosen today?" (bool var)

    # Implications:
    model.Add(bread_rb <= rice_liq) # rice-bread => liquid-rice
    model.Add(bread_rb <= starter_df) # rice-bread => deep-fried starter

    if bread_rb_lits and rice_liq_lits:
        model.Add(starter_df <= bread_rb) # deep-fried starter => rice-bread
        # ONLY when both are available; otherwise this would be infeasible

    model.Add(vegdry_any <= rice_liq) # deep-fried veg-dry => liquid-rice
    model.Add(vegdry_any <= bread_rb) # deep-fried veg-dry => rice-bread

# Weekly limits:
model.Add(sum(bread_rb_day) <= 1)
model.Add(sum(vegdry_df_day) <= 1)
```

■ Analogy

Think of it like a meal kit: rice-bread + liquid-rice + deep-fried starter form a recipe set that goes together. The solver can't pick "just one" without auto-selecting the others. And once you've served this combo this week, you can't serve it again.

■ Key Takeaway

The coupling rule shows how CP-SAT can express "if A then B" logic via inequality constraints ($A \leq B$ means "if $A=1$ then $B=1$ "). Most complex business logic boils down to pairs of these inequalities.

Chapter 26 — The Curd Side Rule

Indian meals always pair a yogurt-based side with rice. The rule decides: plain curd (yogurt) or raita (spiced yogurt)?

- If it's a biryani day -> raita.
- If today's rice is a pulao (subcategory check) -> raita.
- Otherwise -> plain curd.

```
def apply(self, model, ...):
    for di, _ in enumerate(dates):
        rice_pulao_lits = [v for ... if sub_category in pulao_subcats]
        rice_is_pulao = NewBoolVar('rice_is_pulao_{di}')
        link_any(model, rice_pulao_lits, rice_is_pulao)

        curd_lits = [v for ... if sub_category == 'curd']
        raita_lits = [v for ... if is_raita == 1 or 'raita' in sub_category]

        curd_is_curd = NewBoolVar(...); link_any(model, curd_lits, curd_is_curd)
        curd_is_raita = NewBoolVar(...); link_any(model, raita_lits, curd_is_raita)

        if day_type == 'biryani':
            model.Add(curd_is_raita == 1)
        else:
            model.Add(curd_is_raita == 1).OnlyEnforceIf(rice_is_pulao)
            model.Add(curd_is_curd == 1).OnlyEnforceIf(rice_is_pulao.Not())
```

The **OnlyEnforceIf** clauses are CP-SAT's way of expressing conditional constraints: "the equation only has to hold WHEN rice_is_pulao is true."

Chapter 27 — Premium and Unique Items

Two simple but important rules.

PremiumMenuRule

Three constraints in one rule:

```
# 1. Per day cap (max 1 premium item per day)
for di in dates:
    lits = day_premium_vars[di]
    model.Add(sum(lits) <= self.max_per_day)
    model.Add(sum(lits) == prem_day_var) # tracks which days have premium

# 2-3. Per week min and max (between 1 and 2 premium days):
total = sum(prem_day_vars)
model.Add(total >= self.min_per_horizon) # at least 1
model.Add(total <= self.max_per_horizon) # at most 2
```

The `min_per_horizon=1` ensures every week has at least one premium item — a marketing decision: clients pay for variety and value, and one premium dish per week feels like effort. The `max_per_horizon=2` controls food costs.

UniqueltemsMenuRule

“Each item appears at most once across the planning horizon.” We track all switches per item base name and add a sum constraint:

```
def apply(self, model, ...):
    item_to_vars = context.get('item_to_vars', {})
    repeatable = REPEATABLE_ITEM_BASES # {'curd'}
    for item_base, vars_ in item_to_vars.items():
        if item_base not in repeatable:
            model.Add(sum(vars_) <= 1)
```

Curd is the one exception — it's served as a side every day in many configurations. The exclusion list lives in **`src/constants.py`**.

Chapter 28 — Cuisine and Non-Veg Rules

CuisineMenuRule

"Italian must appear on Wednesdays and Thursdays." This is a fine-grained per-day cuisine forcing. For broad cuisine filtering (south, north, Chinese), **ThemeSlotFilterRule** is preferred — **CuisineMenuRule** is for narrower cases:

```
def apply(self, model, ...):
    for di, d in enumerate(dates):
        if self.days_of_week and d.strftime('%A').lower() not in self.days_of_week:
            continue
        for cell in cells:
            if cell.d_idx != di or cell.base_slot in EXEMPT_FROM_CUISINE:
                continue
            cuisine_lits = [v for v, row in zip(cell.x_vars, cell.cand_rows)
                           if row[cuisine_col] == self.cuisine_family]
            if cuisine_lits:
                model.Add(sum(cuisine_lits) >= 1) # at least one Italian
```

NonvegBiryaniWeeklyRule

"At most one non-veg biryani per week."

```
def apply(self, model, ...):
    biryani_day_vars = []
    for di in range(len(dates)):
        nv_cells = [c for c in cells if c.d_idx == di and c.base_slot == 'nonveg_main']
        biryani_lits = [v for c in nv_cells for v, r in zip(c.x_vars, c.cand_rows)
                       if int(r.get('is_nonveg_biryani', 0)) == 1]
        if biryani_lits:
            day_has = NewBoolVar(...)
            link_any(model, biryani_lits, day_has)
            biryani_day_vars.append(day_has)
    if biryani_day_vars:
        model.Add(sum(biryani_day_vars) <= self.max_per_week)
```

NonvegDryPreferenceRule

"If a client has 2 nonveg slots per day (slot_num >= 2), prefer dry items in slot 2 to balance against the gravy in slot 1." This is a pre-filter, not a hard rule:

```
def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
    slot_num = ctx.get('slot_num')
    if base_slot != 'nonveg_main' or not slot_num or slot_num < 2:
        return pool
    # Prefer dry items first; fall back to gravy if no dry available
    if 'is_nonveg_dry' in pool.columns:
```

```
dry_pool = pool[pool['is_nonveg_dry'] == 1]
if len(dry_pool) > 0:
    return dry_pool
if 'is_nonveg_gravy' in pool.columns:
    gravy_pool = pool[pool['is_nonveg_gravy'] == 1]
    if len(gravy_pool) > 0:
        return gravy_pool
return pool
```

■ Key Takeaway

Cuisine and non-veg rules show off the full pattern toolkit: hard min constraints, weekly caps, per-slot-number pre-filters with fallback chains. Most new business logic can be assembled from these patterns.

Part G — Custom Per-Client Rules

The cafeteria serves 18 different companies, each with unique demands. Custom rules let you express “Tekion bans mushroom” or “Stripe wants 2 nonveg items per day” without hardcoding them into the main system. This part shows how.

Chapter 29 — How Custom Rules Are Stored and Loaded

Storage: `client_rules.json`

Every per-client rule lives in `data/configs/client_rules.json`:

```
{
  "Tekion": [
    {
      "name": "tekion_no_mushroom",
      "type": "ingredient_ban",
      "ingredients": ["mushroom"]
    },
    {
      "name": "tekion_liquid_rice_once",
      "type": "item_frequency",
      "base_slot": "rice",
      "selector": { "flag": "is_liquid_rice" },
      "min_per_week": 1,
      "max_per_week": 1
    },
    {
      "name": "tekion_nonveg_mwf",
      "type": "slot_day_restriction",
      "base_slot": "nonveg_main",
      "allowed_weekdays": ["mon", "wed", "fri"]
    }
  ],
  "AnotherClient": [
    ...
  ]
}
```

It's plain JSON — no code. An admin can add or modify rules without touching Python. Each rule has a **name** (for logging), a **type** (one of three options), and type-specific configuration.

Loading: `load_for_client`

In `menu_rule_loader.py`:

```

def load_for_client(self, client_name, generic_rules):
    path = Path(CLIENT_RULES_CONFIG_PATH)
    if not path.exists():
        return list(generic_rules) # no custom rules file

    with open(path, 'r') as f:
        blob = json.load(f)

    client_block = blob.get(client_name)
    if not client_block:
        return list(generic_rules) # this client has no custom rules

    extra = []
    for rule_cfg in client_block:
        rule = self._create_rule(rule_cfg)
        if rule and rule.validate_config():
            extra.append(rule)
        else:
            _log_invalid_rule(rule, rule_cfg, scope=...)

    return list(generic_rules) + extra

```

■ ■ Important

Custom rules are loaded fresh on EVERY request — there's no in-process cache. This means an admin can edit `client_rules.json` and changes apply to the next request immediately. Generic rules ARE cached (loaded once at startup) because they change rarely.

Application: in `api/app.py`

```

def _rules_and_skip_for_client(client_name, dates):
    generic = _get_menu_rules() # cached
    loader = MenuRuleLoader()
    rules = loader.load_for_client(client_name, generic)

    skip_cells = set()
    for rule in rules:
        if hasattr(rule, 'compute_skip_cells'):
            skip_cells |= rule.compute_skip_cells(dates)
    return rules, skip_cells

```

■ Key Takeaway

Custom rules = JSON file + per-request load + same `BaseMenuRule` interface as generic rules. The solver never knows the difference between “built-in” and “custom”; — it just runs every rule in its list.

Chapter 30 — Ingredient Ban Rule

The simplest custom rule. “Don't serve dishes with these ingredients.” It is pure pre-filter.

Configuration

```
{
  "name": "tekion_no_mushroom",
  "type": "ingredient_ban",
  "ingredients": ["mushroom", "babycorn"]
}
```

Implementation

```
class IngredientBanRule(BaseMenuRule):
    def __init__(self, rule_config):
        super().__init__(rule_config)
        raw = rule_config.get('ingredients', [])
        self.banned = {i.strip().lower() for i in raw if isinstance(i, str)}

    def validate_config(self):
        return (
            isinstance(self.config.get('ingredients'), list)
            and len(self.banned) > 0
        )

    def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
        if (len(pool) == 0 or 'key_ingredient' not in pool.columns
            or not self.banned):
            return pool
        ki = pool['key_ingredient'].astype(str).str.strip().str.lower()
        return pool[~ki.isin(self.banned)]

    def apply(self, model, variables, menu_data, context):
        pass # all filtering happens in pre_filter_pool
```

Important: matching is exact, not substring

Banning “corn” will **not** remove items where key_ingredient is “babycorn”. The match is on the full normalized value:

```
banned = {'corn'}
ki = 'babycorn'
'babycorn' in {'corn'} # False - substring isn't checked
```

If you want to ban both, list them both: [“corn”, “babycorn”].

Chapter 31 — Item Frequency Rule

"Liquid rice (any item with is_liquid_rice=1) must appear exactly 1 day per week." The frequency rule expresses minimum and maximum weekly day counts for any selector.

Configuration: the four selector types

```
# Selector by FLAG (boolean column)
{
  "selector": { "flag": "is_liquid_rice" },
  "min_per_week": 1,
  "max_per_week": 1
}

# Selector by SUB_CATEGORY
{
  "selector": { "sub_category": "south_one_pot_rice" },
  "max_per_week": 2
}

# Selector by ITEM (exact match)
{
  "selector": { "item": "khichdi" },
  "max_per_week": 1
}

# Selector by KEY_INGREDIENT
{
  "selector": { "key_ingredient": "paneer" },
  "max_per_week": 3
}
```

Exactly ONE selector key is required. Mixing them is invalid (the rule's `validate_config` returns `False`).

Implementation

```
def apply(self, model, variables, menu_data, context):
    cells = context.get('cells', [])
    dates = context.get('dates', [])
    link_any = context.get('link_any_fn')

    day_vars = []
    for di in range(len(dates)):
        # Find cells matching the optional base_slot filter
        day_cells = [c for c in cells if c.d_idx == di
                     and (self.base_slot is None or c.base_slot == self.base_slot)]

        # Collect every switch whose row matches the selector
        matching_lits = [
            v for c in day_cells for v, r in zip(c.x_vars, c.cand_rows)]
```

```

        if self._row_matches(r)
    ]

    if matching_lits:
        day_has = model.NewBoolVar(f'{self.name}_day_{di}')
        link_any(model, matching_lits, day_has) # day_has = OR of matching
        day_vars.append(day_has)

    if not day_vars:
        return

    if self.max_per_week is not None:
        model.Add(sum(day_vars) <= self.max_per_week)

    if self.min_per_week is not None:
        model.Add(sum(day_vars) >= self.min_per_week)

```

Day-level vs occurrence-level counting

The constraint counts *days*, not *occurrences*. If a client has 2 rice slots per day and both pick liquid rice, that still counts as 1 liquid-rice day. For most slots with count=1 this distinction doesn't matter.

Chapter 32 — Slot Day Restriction Rule

"Don't serve nonveg_main on Tuesdays and Thursdays." The third kind of custom rule: it skips entire (date, slot) cells before they are created.

Configuration

```
{
  "name": "tekion_nonveg_mwf",
  "type": "slot_day_restriction",
  "base_slot": "nonveg_main",
  "allowed_weekdays": ["mon", "wed", "fri"]
}
```

The allowed weekdays accept multiple formats: "mon", "Monday", "MONDAY" — all are normalized to weekday index 0.

Implementation

```
class SlotDayRestrictionRule(BaseMenuRule):
    def __init__(self, rule_config):
        super().__init__(rule_config)
        self.base_slot = rule_config.get('base_slot', '')
        raw_days = rule_config.get('allowed_weekdays', [])
        self.allowed_weekdays = set()
        for tok in raw_days:
            if isinstance(tok, str):
                idx = _WEEKDAY_TOKENS.get(tok.strip().lower())
                if idx is not None:
                    self.allowed_weekdays.add(idx)

    def compute_skip_cells(self, dates):
        return {
            (d, self.base_slot)
            for d in dates
            if d.weekday() not in self.allowed_weekdays
        }

    def apply(self, model, variables, menu_data, context):
        pass # cell skipping is handled by the solver's skip_cells set
```

How skip_cells flows to the solver

```
# In api/app.py:
rules, skip_cells = _rules_and_skip_for_client(...)
# skip_cells now: {(2026-05-05 Tue, 'nonveg_main'),
# (2026-05-07 Thu, 'nonveg_main')}
```

```
# Passed to MenuSolver:
solver = MenuSolver(..., skip_cells=skip_cells)

# In MenuSolver._build_cells:
for di, d in enumerate(dates):
    for slot_id in expanded_slots:
        base = _base_slot(slot_id)
        if (d, base) in self.skip_cells:
            continue # never create this cell
```

■ Key Takeaway

Three custom rule types, three different mechanisms: `ingredient_ban` filters the candidate list, `item_frequency` adds weekly count constraints, `slot_day_restriction` prevents cell creation entirely. Together they cover most per-client requirements.

Chapter 33 — Adding Your Own Custom Rule

Suppose you want to add a new rule type: "color_ban" — never serve any item of a specific color. Here's the recipe:

Step 1 — Create the rule class

Create `src/menu_rules/color_ban_rule.py`:

```
import datetime as dt
import pandas as pd
from typing import Dict, Any

from ortools.sat.python import cp_model
from .base_menu_rule import BaseMenuRule, MenuRuleType
from ..preprocessor.column_mapper import _norm_color

class ColorBanRule(BaseMenuRule):
    """Pre-filter: drop items whose color is in the banned set."""

    def __init__(self, rule_config: Dict[str, Any]):
        super().__init__(rule_config)
        self.rule_type = MenuRuleType.COLOR_BAN # NEW enum value
        raw = rule_config.get('colors', [])
        self.banned = {_norm_color(c) for c in raw if isinstance(c, str)}

    def validate_config(self):
        return (
            isinstance(self.config.get('colors'), list)
            and len(self.banned) > 0
        )

    def pre_filter_pool(self, pool, date, base_slot, day_type, ctx):
        if (len(pool) == 0 or 'item_color' not in pool.columns
            or not self.banned):
            return pool
        col = pool['item_color'].map(_norm_color)
        return pool[~col.isin(self.banned)]

    def apply(self, model, variables, menu_data, context):
        pass
```

Step 2 — Add the enum value

In `base_menu_rule.py`:

```
class MenuRuleType(Enum):
    # ... existing types ...
    COLOR_BAN = "color_ban"
```

Step 3 — Register in the loader

In `menu_rule_loader.py`:

```
from .color_ban_rule import ColorBanRule

class MenuRuleLoader:
    RULE_CLASSES = {
        # ... existing entries ...
        'color_ban': ColorBanRule,
    }
```

Step 4 — Export from package init

In `menu_rules/__init__.py`:

```
from .color_ban_rule import ColorBanRule

__all__ = [
    # ...
    'ColorBanRule',
]
```

Step 5 — Use it

Add to `data/configs/client_rules.json`:

```
"SomeClient": [
  {
    "name": "no_red_for_diet",
    "type": "color_ban",
    "colors": ["red"]
  }
]
```

Step 6 — Test it

Add a test file `tests/test_color_ban_rule.py` following the pattern of `test_menu_rules.py`. Test `validate_config()`, `pre_filter_pool()` with matching items, no-op cases, and edge cases (empty pool, missing column).

■ Tip

If your new rule needs to add CP-SAT equations instead of pre-filtering, follow the pattern in `ItemFrequencyRule`. If it needs to skip cells entirely, follow `SlotDayRestrictionRule` and add a `compute_skip_cells` method.

■ Key Takeaway

Adding a custom rule = subclass BaseMenuRule + register in loader + add JSON config + write tests. Six small steps. The framework does the rest. This is the kind of extension that takes 30 minutes and doesn't risk breaking anything else.

Part H — The Web Layer

The Flask API is the bridge between the Streamlit UI and the solver. It enforces auth, rate limits, validation, and concurrency. Each endpoint is a thin coordinator — it does not contain business logic, it just wires the right pieces together.

Chapter 34 — Flask API Overview

All 14 endpoints live in **api/app.py**:

Method	Path	Purpose
POST	/auth/login	Exchange credentials for a bearer token
GET	/auth/whoami	Validate token, return principal
GET	/health	Liveness + readiness check
GET	/metrics	Counter snapshot
GET	/clients	List client names
POST	/plan	Generate a menu plan
POST	/regenerate	Regenerate selected cells
POST	/save	Persist plan to history
POST	/validate-pools	Dry-run pool sizes
GET	/editor-metadata	Slot/theme metadata for the editor
GET	/client-config/<name>	Read a client's config
PUT	/client-config/<name>	Update a client's config
POST	/client	Create a client
DELETE	/client/<name>	Delete a client

Decorator stack

Most protected endpoints have three decorators:

```
@app.route('/api/v1/plan', methods=['POST'])
@require_api_auth() # validates Bearer token, populates g.api_user
@rate_limit("plan") # token-bucket rate limit per principal
@solver_gate # queue if 2 already running, max 8 queued
def plan_menu():
    ...
```

The order matters: auth runs first (so the rate-limit key uses `g.api_user.email`), then rate limit (so a flood is rejected before we hold a solver slot), then solver gate (the actual queueing).

Chapter 35 — The /plan Endpoint, End-to-End

We covered the high-level flow in Chapter 21. Now let's read the actual handler code, which is shorter than you'd expect:

```
@app.route('/api/v1/plan', methods=['POST'])
@require_api_auth()
@rate_limit("plan")
@solver_gate
def plan_menu():
    try:
        inputs = _prepare_solver_inputs(request.get_json() or {})

        pool_warnings = _validate_pools(
            inputs.pools, inputs.cfg, inputs.rules,
            inputs.weekday_dates, skip_cells=inputs.skip_cells,
        )

        solver = MenuSolver(
            pools=inputs.pools, solver_config=inputs.cfg,
            menu_rules=inputs.rules,
            banned_by_date=inputs.banned,
            ricebread_ban_day=inputs.rb_ban,
            recent_sigs=inputs.recent_sigs,
            skip_cells=inputs.skip_cells,
        )

        week_plan, plan_dates = solver.solve()

        formatter = SolutionFormatter(week_plan, plan_dates,
                                     theme_map=inputs.client_cfg.theme_map)

        response = {
            'success': True,
            'message': f'Menu plan generated for {inputs.client_name}',
            'solution': formatter.to_dict(),
        }
        if pool_warnings:
            response['pool_warnings'] = pool_warnings
        if solver.rule_failures:
            response['rule_warnings'] = solver.rule_failures
            _count_rule_failures(solver.rule_failures)
        metrics.incr('plan_requests_total', outcome='success')
        return jsonify(response)

    except (ValueError, KeyError) as e:
        return jsonify({'success': False, 'error': str(e)}), 400
    except RuntimeError as e:
        metrics.incr('plan_requests_total', outcome='solver_error')
        metrics.incr('solver_failures_total')
        return jsonify({'success': False, 'error': str(e)}), 500
    except Exception:
        return _internal_error_response(500)
```

Three exception classes, three different responses. **ValueError** and **KeyError** become 400 (caller sent bad input). **RuntimeError** becomes 500 with the error message (solver failed despite valid input). Anything else becomes a generic 500 with no details (security: don't leak internals).

Chapter 36 — /regenerate and /save

/regenerate

User has a generated plan, doesn't like Tuesday's bread and Wednesday's veg dry. They click those cells and hit Regenerate. The frontend sends:

```
{
  "client_name": "Tekion",
  "base_plan": { "2026-05-04": {...}, ... },
  "replace_slots": {
    "2026-05-05": ["bread"],
    "2026-05-06": ["veg_dry"]
  }
}
```

The handler invokes **MenuRegenerator**, which locks every cell NOT in **replace_slots** (sets locked dict so those switches are forced to their existing values) and re-solves only the selected cells. A similarity score in the objective biases the solver toward items resembling the originals.

/save

Saving is a single-shot insert into Supabase — no retry on transient errors, because retrying could double-insert.

```
@app.route('/api/v1/save', methods=['POST'])
@require_api_auth()
def save_plan():
    # ... validate inputs ...
    sig = HistoryManager.compute_week_signature(week_plan, dates, ...)
    HistoryManager().save(week_plan, dates, client_name, week_start, sig,
                          supabase_client=sb,
                          strip_color_fn=strip_color_suffix)
    return jsonify({'success': True, 'message': 'Plan saved to history'})
```

UNIQUE INDEXES on **menu_history** (client_name, service_date, slot, item_base) and **week_signatures** (client_name, week_start, week_signature) provide a safety net: even if a duplicate insert somehow reaches the DB, the index prevents it.

Chapter 37 — Validation and Error Handling

Three layers of error response, each with a different audience:

Status	When	Body
400	Caller sent bad input	{'success': False, 'error': '<specific message>'}
401	Missing/expired token	{'success': False, 'error': 'Token expired'}
403	Insufficient role	{'success': False, 'error': 'Insufficient role'}
409	Optimistic concurrency conflict	{'success': False, 'current_version': N, ...}
429	Rate limit hit	{'success': False, 'error': 'Too many requests', 'retry_after_seconds': N}
500	Solver couldn't find a plan	{'success': False, 'error': '<RuntimeError message>'}
500 (generic)	Unexpected exception	{'success': False, 'error': 'Internal server error', 'request_id': '...'}
503	Solver queue full or Supabase down	{'success': False, 'error': 'Server at capacity'}
504	Request timed out in queue	{'success': False, 'error': 'Request timed out waiting'}

■ ■ Important

The generic 500 NEVER leaks exception details — that could expose internal hostnames or schema names. Instead, it includes a **request_id** the operator can grep in the access log to find the real exception with full traceback.

Part I — Concurrency and Performance

Solving menu plans is CPU-heavy. Naive parallelism would overload the server. This part covers how the codebase keeps things responsive under load.

Chapter 38 — The Solver Gate

Defined in **api/concurrency.py**. The gate caps concurrent solves at 2 (configurable via `MAX_RUNNING`) and queues up to 8 more (`MAX_QUEUED`). Beyond that, requests are rejected with 503.

```
MAX_RUNNING = 2 # at most 2 solves executing simultaneously
MAX_QUEUED = 8 # max requests waiting
WORKERS_BY_LOAD = { # CP-SAT workers per solve based on load
    1: 9, # solo solve gets 9 workers
    2: 5, # two parallel solves: 5 workers each
}
QUEUE_TIMEOUT = 300 # seconds queued before giving up
```

The clever bit: when only one solve is running, it gets 9 CP-SAT workers (~all available cores). When a second solve starts, the running one shares CPU and both reduce to 5 workers each. Tuned for 1 GB RAM.

The decorator

```
@functools.wraps(fn)
def wrapper(*args, **kwargs):
    global _running

    with _lock:
        if _running < MAX_RUNNING:
            _running += 1
            my_event = None # I run immediately
        elif len(_queue) < MAX_QUEUED:
            my_event = threading.Event() # I wait
            _queue.append(my_event)
        else:
            return jsonify({'error': 'capacity'}), 503

    if my_event is not None:
        got_slot = my_event.wait(timeout=QUEUE_TIMEOUT)
        # ... handle race conditions ...

    try:
        return fn(*args, **kwargs) # do the work
```

```
finally:
    with _lock:
        _running -= 1
        while _running < MAX_RUNNING and _queue:
            next_event = _queue.popleft()
            _running += 1
            next_event.set() # wake the next waiter
```

■ Key Takeaway

Backpressure protects the system. Better to return 503 quickly than accept work the server can't actually finish. The Streamlit client auto-retries on 503 with jitter, so transient queue fullness is invisible to the user.

Chapter 39 — Rate Limiting

Token-bucket rate limiter in `api/rate_limit.py`. Each principal (by email) has their own bucket. Buckets refill over time.

Configured limits

Bucket	Capacity	Refill rate	Why
plan	10	10/min	An admin clicking shouldn't hit this; an automated client will.
regenerate	20	20/min	Higher because regen is cheaper and more interactive.
login_ip	30	30/min	Loose: corporate NAT means many users share an IP.
login_email	5	5/min	Tight: protects individual accounts from credential stuffing.

Why login has TWO buckets

Login is special. We check BOTH `login_ip` AND `login_email` *before* running `bcrypt`. `bcrypt` takes ~250ms per call. Without rate limiting, a brute-force script could saturate the threadpool with hundreds of slow password checks per second.

```
# In api/auth.py, before bcrypt:
ip_key = f"ip:{request.remote_addr or 'unknown'}"
email_key = f"email:{email}"
for limit_name, key in (("login_ip", ip_key), ("login_email", email_key)):
    rejection = check_rate_limit(limit_name, key)
    if rejection is not None:
        return rejection # 429 before bcrypt

# Only NOW do we check the password:
user = AuthManager().authenticate(email, password)
```

Chapter 40 — Caching at Every Layer

Caching boundaries in this codebase are deliberate. Some things are cached forever, some per-request, some not at all.

What	Cache duration	Why
Menu Excel data + pools	Process lifetime	Excel doesn't change without restart.
Generic rules (indian_menu_rules.json)	Process lifetime	Stable config.
Custom rules (client_rules.json)	None — fresh per request	Admin edits should apply immediately.
Client config (Supabase)	Per-request via Flask g	Live edits + minimize round trips within one request.
MenuApiClient instance (Streamlit)	23 hours via @st.cache_resource	Reuse the requests.Session pool.
list_clients result (Streamlit)	60 seconds via @st.cache_data	Sidebar dropdown shouldn't re-fetch every click.

■ Tip

When you see “Why doesn't my edit show up?”, check whether the data lives behind a cache. Generic rules need a process restart; custom rules apply on the next request; client config applies on the next request after the editor's PUT.

Part J — Authentication and Security

Security in this codebase is layered: bcrypt password hashing, signed bearer tokens, role-based access, optimistic concurrency for editor writes, and a kill-switch for legacy hash migration.

Chapter 41 — Roles and Permissions

Three roles, defined in `user_authentication/models.py`:

```
ROLE_SUPER_ADMIN = "super_admin"
ROLE_ADMIN = "admin"
ROLE_USER = "user"

CAN_CREATE_ROLES = {
    ROLE_SUPER_ADMIN: [ROLE_SUPER_ADMIN, ROLE_ADMIN, ROLE_USER],
    ROLE_ADMIN: [ROLE_USER],
    ROLE_USER: [],
}
```

Hierarchy: `super_admin > admin > user`. Higher roles inherit lower ones' permissions. The `require_api_auth(min_role=...)` decorator enforces this on each endpoint.

Chapter 42 — Password Hashing and Bearer Tokens

Passwords: bcrypt with legacy SHA-256 migration

New passwords are bcrypt-hashed at cost factor 12. Legacy SHA-256 hashes (from before the migration) are still verified, but on a successful login they're transparently rehashed to bcrypt. A metric (`legacy_sha256_verifications_total`) lets ops watch the migration drain to zero.

```
def _verify_password(password, stored):
    if stored.startswith('$2'):
        return bcrypt.checkpw(password.encode(), stored.encode())
    if _is_legacy_sha256(stored):
        if _legacy_disabled(): # kill-switch
            return False
        ok = _verify_legacy_sha256(password, stored)
        _incr_metric('legacy_sha256_verifications_total',
                    result='success' if ok else 'fail')
        return ok
    return False
```

Bearer tokens: signed with **API_SECRET_KEY**

On login, the API issues a signed token via **itsdangerous**. The token contains email, role, and profile_name in its payload. It's self-contained — no server-side session table — and time-limited (default 24 hours).

```
def issue_token(email, role, profile_name):
    return _serializer().dumps(
        {"email": email, "role": role, "profile_name": profile_name},
    )

def decode_token(token):
    return _serializer().loads(token, max_age=API_TOKEN_TTL_SECONDS)
```

Rotating **API_SECRET_KEY** invalidates every token in flight — the escape hatch if a token leaks.

Chapter 43 — Cookie-Based Session Restoration

Streamlit's `session_state` is in-memory and dies on hard refresh or new tab. Without persistence, every page load would force a re-login. The fix: store the bearer token in a browser cookie.

Why streamlit-cookies-controller, not extra-streamlit-components

Streamlit custom components run inside an iframe at a different origin than the main app (e.g. on Streamlit Cloud). `extra-streamlit-components.CookieManager` writes cookies via `document.cookie` from inside the iframe, so the cookie lands on the iframe's origin and the browser never sends it back to the main app.

`streamlit-cookies-controller` uses `postMessage` to ask the parent window to set the cookie, which lands it on the correct origin. That's the bug they hit in production and why this library was chosen.

The async warmup dance

The cookie component is asynchronous: `getAll` returns `{}` on the first script run because the JS-to-Python round-trip hasn't completed yet. The auth gate handles this with warmup retries:

```
def _try_restore_session_from_cookie():
    cookies = get_all_cookies()
    if not cookies: # warming up, OR genuinely no cookie
        attempts = st.session_state.get(_COOKIE_WARMUP_KEY, 0)
        if attempts < _COOKIE_WARMUP_MAX_ATTEMPTS:
            st.session_state[_COOKIE_WARMUP_KEY] = attempts + 1
            time.sleep(0.25)
            st.rerun() # try again next render
        return False # budget exhausted, treat as no cookie

    token = cookies.get(COOKIE_NAME)
    if not token:
        return False

    # Validate via /whoami (no Supabase round trip – pure HMAC verify)
    info = MenuApiClient(_BACKEND_URL, token=token).whoami()
    login_user(User(...), token=token)
    return True
```

■ Key Takeaway

Cookie restoration was the trickiest UI bug they solved. The lesson: when integrating async browser features with Streamlit, expect to retry across multiple reruns until the JS layer catches up.

Chapter 44 — Optimistic Concurrency

Two admins editing the same client simultaneously could last-write-wins silently — one's changes vanish without warning. Optimistic concurrency prevents this without locking.

How it works

```
# Schema:
CREATE TABLE clients (
    name TEXT PRIMARY KEY,
    menu_category TEXT NOT NULL,
    version INT NOT NULL DEFAULT 1, -- the optimistic counter
    ...
);

# GET /client-config/Tekion -> {"version": 3, ...}, ETag: "3"

# Admin A edits and PUTs with version=3 -> success, version becomes 4
# Admin B (still has version=3) tries to PUT -> conditional UPDATE matches 0 rows
# -> 409 Conflict with current_version=4
# Admin B refreshes, sees version=4 + A's changes, redoes their edit
```

```
def bump_version_if_matches(self, name, expected):
    new_version = expected + 1
    result = (self._sb.table('clients')
              .update({'version': new_version})
              .eq('name', name)
              .eq('version', int(expected)) # the conditional check
              .execute())
    if not result.data:
        current = self.get_client_version(name)
        raise ConcurrentEditError(..., current_version=current)
    return new_version
```

■ Key Takeaway

Optimistic concurrency: include a version counter in every read, send it back on every write, fail loudly if someone bumped it in the meantime. Standard HTTP idiom (ETag + If-Match header) plus a convenience JSON-body alternative.

Part K — Database Layer

Supabase (managed Postgres) stores everything except the menu ontology (which is Excel). Nine tables in total.

Chapter 45 — Supabase Schema Explained

Table	Purpose
clients	Client registry: name, menu_category, version (optimistic concurrency)
menu_categories	Slot templates: name, slots[]
slot_count_overrides	Per-client slot counts (e.g., Rippling: veg_dry x 2)
theme_overrides	Per-client day-to-theme mapping
app_settings	Misc tunables (core_min_one_slots, constant_slots, fallback)
users	Auth: email, profile_name, password_hash, role
menu_history	One row per (client, date, slot, item) served
week_signatures	One row per saved week plan (the signature hash)

Chapter 46 — The Live-Read Pattern

ClientConfigLoader reads Supabase on every call — there is no in-memory cache. Why? So an admin's edit via the customisation editor is reflected on the next plan request, no restart needed. The cost: a few extra Supabase round trips per request.

Per-request memoization on Flask's **g** object collapses the intra-request round trips:

```
def _cached_on_g(key, compute):
    if not has_request_context():
        return compute()
    cache = getattr(g, '_clientcfg_cache', None)
    if cache is None:
        cache = {}
        g._clientcfg_cache = cache
    if key not in cache:
        cache[key] = compute()
    return cache[key]
```

Chapter 47 — Schema Migration Safety

Three SQL files in **scripts/** set up the database. All three are idempotent (CREATE TABLE IF NOT EXISTS, ALTER TABLE ... ADD COLUMN IF NOT EXISTS) so re-running is a no-op.

If a deployment runs new code against an unmigrated database (e.g., the **clients.version** column is missing), the loader logs a clear ERROR pointing at the migration script and falls back to non-concurrent writes. The /health endpoint reports schema drift in its JSON body, while still returning HTTP 200 (the app is degraded but not broken).

Part L — Streamlit Frontend

The Streamlit app is a thin renderer that consumes JSON from the Flask API. Most of its complexity is around session state, cookie rehydration, and CSS theming — not business logic.

Chapter 48 — The Main App Flow

app.py top-to-bottom:

- Streamlit page config (must be first command).
- Validate required env vars; show error and stop if missing.
- Auto-spawn Flask backend on a free local port (5000-5004).
- `init_auth_state()` and try to restore session from cookie.
- If unauthenticated: render login form and stop.
- Otherwise: render sidebar (client picker, date, days slider).
- If user clicks Generate: call `client.plan()`, store result in `session_state`.
- If a plan is in `session_state`: render the colored menu table + action buttons.

Auto-spawning Flask

```
def _ensure_backend_running():
    port = pick_backend_port() # 5000-5004, first bindable
    url = f"http://localhost:{port}"
    if health_check(port):
        return url # already running
    if "flask_started" not in st.session_state:
        t = threading.Thread(target=_start_flask_backend, args=(port,), daemon=True)
        t.start()
        st.session_state.flask_started = True
    for _ in range(20):
        if health_check(port):
            return url
        time.sleep(0.5)
    raise RuntimeError("Backend did not become healthy")
```

Chapter 49 — Customisation Editor

The **customisation/** folder has four files that compose the editor view: `main.py` (dispatcher), `slot_editor.py` (toggle categories), `multi_slot_editor.py` (set count per slot), `theme_editor.py` (per-day theme override).

Two modes: Select Existing (edit a client) and Create New (build a client from scratch). Both produce a config dict that gets PUT to `/api/v1/client-config/<name>`.

Chapter 50 — User Manager UI

Role-gated admin page. Renders a list of users with role badges and a delete button. Below it is a Create User form whose role options are filtered to **creatable_roles** for the current user (super_admin can create any role; admin can create only users).

Part M — Operations and Observability

Production readiness: structured logs, in-process metrics, health checks that distinguish liveness from schema drift.

Chapter 51 — Structured Logging

Set **LOG_FORMAT=json** to emit one JSON object per log record. Each line carries timestamp, level, logger name, message, request_id, plus any caller-supplied **extra** fields.

request_id is a Python ContextVar, set in **before_request** and stamped onto every log record by a logging Filter. It also goes into the **X-Request-ID** response header so callers can correlate.

```
class JsonFormatter(logging.Formatter):
    def format(self, record):
        payload = {
            "ts": ... ISO8601 ...,
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
            "request_id": getattr(record, "request_id", "-"),
        }
        if record.exc_info:
            payload["exc"] = self.formatException(record.exc_info)
        # ... include caller's extra= fields ...
        return json.dumps(payload, separators=(",", ":"))
```

Chapter 52 — In-Process Metrics

Tiny counter library in **api/metrics.py**. No prometheus_client, no statsd — just a thread-safe dict of counters with Prometheus-style label keys. The /metrics endpoint returns a snapshot.

```
metrics.incr("plan_requests_total", outcome="success")
metrics.incr("rule_failures_total", rule="cuisine")

# Snapshot looks like:
{
    'plan_requests_total{outcome="success"}': 14,
    'rule_failures_total{rule="cuisine"}': 2,
    'solver_failures_total': 1,
}
```

Chapter 53 — Health Checks

GET /api/v1/health returns 200 when healthy, 503 when degraded. Either way the body has actionable info: supabase_reachable, schema status (ok / drift_detected), uptime, queue stats.

Schema drift (e.g., the **version** column missing) is reported in the body but does NOT flip HTTP status. The app keeps serving via fallback paths; an operator reading logs sees the loud ERROR and runs the migration.

Part N — Testing and CI

477 tests, ~84% coverage. CI runs them on every PR plus ruff and bandit. The slow-tests job runs only on push-to-main and manual dispatch so PR feedback stays fast.

Chapter 54 — Test Architecture

File	What it tests
test_solver.py + test_rule_constraints.py	Solver + constraint integration
test_menu_rules.py	Each rule's validate_config and basic behavior
test_menu_rule_loader.py	JSON deserialization
test_pool_builder.py + test_column_mapper.py	Data pipeline
test_history_manager.py	Cooldowns and signatures
test_client_config*.py	Supabase-backed config
test_api*.py	Flask endpoints
test_auth.py	Auth manager + bcrypt + legacy migration
test_cookie_store.py	Cookie persistence
test_concurrency.py + test_rate_limit.py	Solver gate + rate limits
test_integration_slow.py	@slow: real Excel + real rules + 5-day solve

Chapter 55 — Fake Supabase

tests/fake_supabase.py is an in-memory implementation of the subset of supabase-py the app uses. It supports the chained **.table().select().eq().execute()** API plus inserts, updates, and deletes. About 200 lines of code that replaces a real database for every test.

The **fake_supabase** fixture in **conftest.py** seeds it with one client (Rippling) and a default menu category. Tests can then exercise the API without setting up a real Supabase project.

Chapter 56 — CI Pipeline

`.github/workflows/ci.yml` defines four jobs:

- `pytest` — full unit suite with coverage gate ($\geq 82\%$).
- `ruff` — bug-only ruleset (F and E9): undefined names, syntax errors.
- `bandit` — medium+ severity security findings.
- `slow-tests` — real-Excel integration tests, only on push-to-main and manual dispatch.

Part O — Final Recap and Mental Models

We covered a lot. This last part distills it into a single mental model and answers common questions.

Chapter 57 — The Whole System, One Page

```
USER (Streamlit app.py)
|
v HTTP + Bearer token
FLASK API (api/app.py)
| decorators: auth, rate_limit, solver_gate
|
+---> ClientConfigLoader (Supabase live read)
+---> ExcelReader -> ColumnMapper -> DataCleanser -> PoolBuilder
+---> MenuRuleLoader (generic + per-client custom rules)
+---> HistoryManager (cooldown + signature data from Supabase)
|
v
MENSOLVER (CP-SAT)
|
1. _build_cells: pre-filter pools, sample candidates
2. _build_decision_variables: NewBoolVar per (cell, candidate)
   sum(cell vars) == 1 per cell
3. _add_color_constraints: built-in color rules
4. for rule in menu_rules: rule.apply(model, ..., context)
5. _build_objective: random tiebreaker + soft rule terms
6. solver.Solve() with multi-restart on infeasibility
7. _extract_solution_rows: read which switch is ON per cell
8. _rows_to_week_plan: format with color suffixes + constants
|
v
SOLUTIONFORMATTER -> JSON -> Streamlit table
```

■ Key Takeaway

Eight steps inside MenuSolver. The rest of the codebase exists to feed those eight steps the right inputs and present their outputs. Once you can recite those eight steps, you understand the system.

Chapter 58 — Common Questions and Pitfalls

Q: Why does “edit X in the editor” not show up immediately?

Most edits ARE live (client config has no cache, custom rules read fresh per request). Exceptions: the Streamlit sidebar's client list is cached for 60s — the editor's create/delete handlers call `st.cache_data.clear()` to invalidate it. Generic rules (`indian_menu_rules.json`) ARE cached at process startup; edit them and you need a backend restart.

Q: I added a new rule type. Why is it being ignored?

Three places to register, easy to miss one:

- `MenuRuleType` enum in `base_menu_rule.py`
- `MenuRuleLoader.RULE_CLASSES` dict in `menu_rule_loader.py`
- Export from `menu_rules/__init__.py`

Q: The solver returned INFEASIBLE. What now?

Most of the time, multi-restart hides this — the solver tries 8 different combinations and you only see the failure if all 8 fail. Causes to investigate:

- Tight history cooldown leaving no candidates.
- Theme filter + ingredient ban combining to empty a pool.
- Conflicting custom rules (e.g., `min_per_week` + `slot_day_restriction` colliding).
- Pool warnings in the response (look at `pool_warnings`).

Q: How do I add a new endpoint?

Three steps:

- Add the route in `api/app.py` with the right decorators.
- Add a corresponding method on `MenuApiClient` in `ui/api_client.py`.
- Use it from Streamlit or the editor.

Q: What's the right way to debug a 500?

Generic 500 responses include a `request_id`. Grep that id in the access log; the matching ERROR line a few rows earlier is the real exception with traceback.

■ Key Takeaway

You now have the full picture. The codebase is ~30k lines of Python, but every line fits into the four-floor model and the eight-step solver flow. Hold those two mental models in your head and the rest is just details. Welcome to the team.